"DIABETES PREDICTION USING DIFFERENT MACHINE LEARNING APPROACHES"

The report of the project is submitted to Bangalore North University in partial fulfilment of the requirements for the award of the Degree

BACHELORE OF COMPUTER APPLICATIONS

Submitted By

Navya Shree K N (R2011632)

Under The Guidance of

Anusha Priyadharshini C B.E, M. Tech

Assistant Professor & HOD

Department of Computer Science

॥ श्रद्धाहि परमा गतिः ॥

Department of Computer Science

The National Degree College, Bagepalli

(Affiliated to Bangalore North University, Kolar)

2022-2023

Department of Computer Science

The National Degree College, Bagepalli

(Affiliated to Bangalore North University, Kolar)

॥ श्रद्धाहि परमा गतिः ॥

CERTIFICATE

This is to certify that project work entitled "Diabetes Prediction Using Different Machine Learning Approaches" done by Navya Shree K N (R2011632), in partial fulfilment for the award of Degree in Bachelor of Computer Applications from The National Degree College, Bagepalli.

Project Guide

Anusha Priyadarshini C

Head of the Department

Signature of Principal

ACKNOWLEDGEMENT

I consider it as a privilege to whole heartedly express our gratitude and respect to each one who guided and helped us in the successful completion of our project. I would like to thank **Prof. Imtiyaz Ahmed Mubeen M**, principal for providing the infrastructure and facilities for the completion of our project.

I would also greatly mention the enthusiastic influence and guidance provided by **Mrs. Anusha Priyadharshini .C**, Head of Computer Science Department & my project guide, for her technical guidance, constant encouragement, and support in carrying out our project at college and making our project a great success and her co-operation and encouragement at all movements of our approach.

The satisfaction of that accompany the successful completion of the task would be great but incomplete without the mention of people who made it possible with their constant guidance and encouragement crowns all efforts with success.

I would like to extend our gratitude towards all the faculty members of computer science department for constant support and encouragement in completion of our project.

Finally, we would also like to express a heartfelt gratitude to our parents and as well as well-wishers for their guidance and kind co-operation.

Navya Shree K N **(R2011632)**

ABSTRACT

Recent days Diabetes Mellitus (DM) is major chronic disease many people are facing. Diabetes Mellitus is a disease with inadequate control of blood glucose levels. It is given to the body by insulin that carries out the metabolic tasks that keep people alive. Glucose level imbalance is a sign of diabetes mellitus (DM), a common type of chronic disease. It leads to long-term complications having a negative impact on one's quality of life. The diagnosis of the correct type is sometimes ambiguous, this study aims to utilize, an ensemble Machine Learning (ML)approach to distinguish and predict three types of diabetes called TD1, TD2 and Gestational diabetes based on Pima Indian Diabetes dataset.

The adaption of machine learning in the diabetes prediction and detection provides various promising solutions for the existing problems related to manual diagnosis which painful and time consuming inadequate. This approach also reduces the gestational diagnosis that could be done on the patient and intern this saves the time of the doctors to take a decision about the patient as well as expenses incurred by the patient.

CONTENTS

CHAPTER	NAME	PAGNO
CHAPTER-1	INTRODUCTION	1-3
	1.1 Introduction……01	
	1.2 Existing System……..02	
	1.3 Problem Statement…….....03	
	1.4 Proposed System…….....03	
CHAPTER-2	LITERATURE SURVEY	4-7
	2.1 Literature Survey…….4-6	
	2.2 Functional and Non-Functional Requirements…….6-7	
CHAPTER-3	SYST EM DESIGN & MODELING	8-10
	3.1 System Architecture……....08	
	3.2 Use case Diagrams…….....8-9	
	3.2.1 Use case diagram User……...9	
	3.3 Data flow Diagram……9-10	
	3.3.1 Level 0 data flow diagram……...10	
	3.4.2 Level 1 data flow diagram……...10	
CHAPTER-4	IMPLEMENTATION TECHNOLOGIES	11-34
	4.1 Python Introduction……11-17	
	4.2 Algorithms Used……17-18	

4.3 Artificial Intelligence……18-19

4.4 Machine Learning……19-22

4.5 SDLC Project Methodology……22-34

CHAPTER-5 SYSTEMREQUIREMENTSPECIFICATION 35-36

5.1 System Requirement Specification……..35

5.2 Hardware and Software Requirements……..36

CHAPTER-6 TESTING AND IMPLEMENTATION 37-42

6.1 Software testing introduction……37

6.2 Explanation for SDLC & STLC……37

6.3 Phases of software development……37

6.3.1 Requirement analysis……..37

6.3.2 Design……..37-38

6.3.3 Testing……..38

6.3.4 Maintenance……38

6.4 SDLC Models……38

6.4.1 Water fall model……38

6.4.2 Prototype model……..38

6.4.3 Rapid application development model (RAD)…….38

6.4.4 Spiral model……38

6.4.5 V- Model……39

6.5 STLC (Software Testing Life Cycle) …….39

6.5.1 Test Plan……39

6.5.2 Test Development…….39

6.5.3 Test Execution……39

6.5.4 Analyse Result……39

6.5.6 Defect Tracking…….39-40

6.6 Types of Testing……40

6.6.1 White Box Testing……..40

6.6.2 Black Box Testing……..40

6.6.3 Grey Box Testing……...40

6.7 Level of Testing used in Project……..40

6.7.1 Unit Testing……40

6.7.2 Integration Testing……40-41

6.7.3 System Testing……...41

6.7.4 Functional Testing……41

6.7.5 Alpha Testing……41

6.7.6 Beta Testing……...41

6.8 Unit Testing Cases……..41-42

CHAPTER-7 RESULT AND ANALYSIS 43-57

7.1 Result Analysis Screenshots……...43-53

7.2 Front End Design Screenshots…….54-57

CHAPTER-8 CONCLUSION 58

CHAPTER-9 REFERENCE 59

CHAPTER-1

INTRODUCTION

1.1 Introduction

Diabetes is a chronic disease it is a condition in which the body is incapable of producing insulin. Insulin is a hormone which mainly responsible for maintaining the blood glucose level.

Many factors of diabetes, such as:

- Excessive Body Weight
- Physical Inactivity
- Family History
- High Blood Pressure

Some researchers divided diabetes into Type1, Type2, and Gestational diabetes.

Type1: Type1 diabetes is a condition in which Pancreas produce very little insulin to our body.

Type2: Type2 diabetes occurs when body does not produce sufficient insulin or the body becomes insulin resistant. It occurs according high blood pressure, smoking, sedentary lifestyle, overweight, high fat & cholesterol levels.

Gestational diabetes: It is a type of diabetes which occurs only in pregnancy due to hormonal changes.

The common symptoms of diabetes

- Polydipsia
- Polyphagia
- Sudden Weight Loss
- Weakness
- Obesity
- Visual Blurring

The risk factors of diabetes are...

- Diabetic retinopathy
- Brain stroke
- Diabetic cardiomyopathy

- Diabetic foot
- Diabetic nephropathy
- Reduced blood flow
- Diabetic neuropathy

Applying machine learning and data mining methods in DM research is a key approach to utilizing large volumes of available diabetes-related data for extracting knowledge. The severe social impact of the specific disease renders DM one of the main priorities in medical science research, which inevitably generates huge amounts of data. Undoubtedly, therefore, machine learning and data mining approaches in DM are of great concern when it comes to diagnosis, management, and other related clinical administration aspects. Hence, in the framework of this study, efforts were made to review the current literature on machine learning and data mining approaches in diabetes research.

Abdullah's Data mining methods support health care researchers to retrieve novel knowledge from large health data. With the development of Information Technology, Data mining offers appreciated advantage in diabetes research, which leads to expand or improve health care distribution, increase support for decision –making and improve disease supervision. Saba's no single technique gives highest accuracy or accuracy for all diseases, whereas one classifier provides or shows better performance in each dataset, another method or approach outdoes the others for other diseases.

1.2 Existing System

Different existing data mining procedures and its application were considered or explored. Utilization of machine learning algorithms was connected in various medical data sets. Machine learning strategies have diverse power in different medical data sets. Previously mentioned conventional machine learning techniques gave less exact outcome and results additionally shifts considering the procedures has been utilized for the prediction.

Disadvantages of Existing System

- Different prediction result on different datasets and techniques
- Less Accurate and Effective prediction Result

1.3 Problem Statement

Prior examination and research in this field is not sufficient for early prediction and to get more accurate and efficient results. Therefore, a machine learning model is required to overcome the above stated problem.

1.4 Proposed system

Our proposed strategy focuses on a novel machine learning procedures for Diabetes classification, thus overcoming the existing problem. By utilizing Random Forest (RF), Support Vector Machine (SVM) we will make our model in order to increase the performance and accuracy

Advantages of Proposed System

- model predicts more accurate results.
- Higher performance compares to existing system.

CHAPTER-2

LITERATURE SURVEY

2.1 Literature Survey

1. Prediction and diagnosis of diabetes mellitus — A machine learning approach:

Abstract: Diabetes is a disease caused due of the expanded level of sugar fixation in the blood. Various computerized information systems were outlined utilizing diverse classifiers for anticipating and diagnosing diabetes. Selecting legitimate classifiers clearly expands the exactness and proficiency of the syst4em. Here a decision support system is proposed that uses AdaBoost algorithm with Decision Stump as base classifier for classification. Additionally, Support Vector Machine, Naive Bayes and Decision Tree are also implemented as base classifiers for AdaBoost algorithm for accuracy verification. The accuracy obtained for AdaBoost algorithm with decision stump as base classifier is 80.72% which is greater compared to that of Support Vector Machine, Naive Bayes, and Decision Tree.

2. Diagnosing Diabetes using Data Mining Techniques.

Abstract: Diabetes is a disease which is affecting many people now-a-days. Most of research is happening in this area. In this paper, we proposed a model to solve the problems in existing system in applying data mining techniques namely clustering and classifications which are applied to diagnose the type of diabetes and its severity level for every patient from the data collected. This paper tries to diagnose diabetes based on the 650 patient's data with which we analyse, and identified severity of the diabetes. As part of procedure Simple k-means algorithm is used for clustering the entire dataset into 3 clusters i.e., cluster-0 - for gestational diabetes, cluster-1 for type-1 diabetes (juvenile diabetes), cluster-2 for type-2 diabetes. This clustered dataset was given as input to the classification model which further classifies each patient's risk levels of diabetes as mild, moderate, and severe. Further, performance analysis of different algorithms has been done on this data to diagnose diabetes. The achieved results show the performance of each classification algorithm.

3. Application of machine learning algorithms on diabetic retinopathy

Abstract: Diabatic Retinopathy (DR) is one of the leading causes of sight inefficiency for diabetic patients. The clinical diagnostic results and several outcomes of eye testing methods reviled a set of observations that eases the decision making in the case of diabetic retinopathy for the doctor, therapist.

Machine learning, a branch of artificial intelligence is applied in clinical data analytic as it can detect patterns in data, and then use these uncovered patterns to predict future data or perform some kind of decision making under uncertainty. In case of DR finding the co-relation between the depth of affection and the clinical result is very much critical, as several parameters are need to be taken into consideration for optimal decision making by the therapist. In this paper we have reviewed the performance of a set of machine learning algorithms and verify their performance for a particular DR data set.

4. Analysis of Various Data Mining Techniques to Predict Diabetes Mellitus.

Abstract: Data mining approach helps to diagnose patient's diseases. Diabetes Mellitus is a chronic disease to affect various organs of the human body. Early prediction can save human life and can take control over the diseases. This paper explores the early prediction of diabetes using various data mining techniques. The dataset has taken 768 instances from PIMA Indian Dataset to determine the accuracy of the data mining techniques in prediction. The analysis proves that Modified J48 Classifier provide the highest accuracy than other techniques.

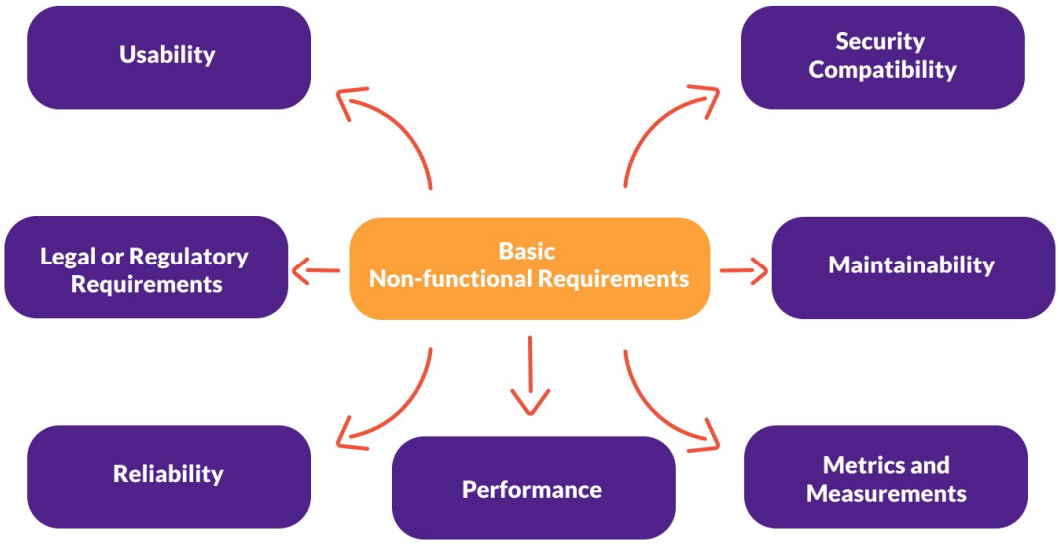
5. Study of machine learning algorithms for special disease prediction using principal of component analysis

Abstract: The worldwide study on causes of death due to heart disease/syndrome has been observed that it is the major cause of death. If recent trends are allowed to continue, 23.6 million people will die from heart disease in coming 2030. The healthcare industry collects large amounts of heart disease data which unfortunately are not "mined" to discover hidden information for effective decision making. In this paper, study of PCA has been done which finds the minimum number of attributes required to enhance the precision of various supervised machine learning algorithms. The purpose of this research is to study supervised machine learning algorithms to predict heart disease. Data mining has number of important techniques like categorization, preprocessing. Diabetic is a life-threatening disease which prevent in several urbanized as well as emergent countries like India. The data categorization is diabetic patients' datasets which is developed by collecting data from hospital repository consists of 1865 instances with dissimilar attributes. The examples in the dataset are two categories of blood tests, urine tests. In this research paper we discuss a variety of algorithm approaches of data mining that have been utilized for diabetic disease prediction. Data mining is a well-known practice used by health organizations for classification of diseases such as diabetes and cancer in bioinformatics research.

2.2 Functional requirements

- Collect Data from dataset
- Data visualization
- Data Splitting into training and testing
- Train a model using training data
- Prediction using testing data

Non-functional requirements: Non-functional requirements describe how a system must behave and establish constraints of its functionality. This type of requirements is also known as the system's quality attributes. Attributes such as performance, security, usability, compatibility are not the feature of the system, they are a required characteristic. They are "developing" properties that emerge from the whole arrangement and hence we can't compose a particular line of code to execute them. Any attributes required by the customer are described by the specification. We must include only those requirements that are appropriate for our project.



Some Non-Functional Requirements are as follows:

- **Reliability**

  The structure must be reliable and strong in giving the functionalities. The movements must be made unmistakable by the structure when a customer has revealed a couple of enhancements. The progressions made by the Programmer must be Project pioneer and in addition the Test designer.

- **Maintainability**

  The system watching and upkeep should be fundamental and focus in its approach. There should not be an excess of occupations running on diverse machines such that it gets hard to screen whether the employments are running without lapses.

- **Performance**

  The framework will be utilized by numerous representatives all the while. Since the system will be encouraged on a single web server with a lone database server outside of anyone's ability to see, execution transforms into a significant concern. The structure should not capitulate when various customers would use everything the while. It should allow brisk accessibility to every piece of its customers. For instance, if two test specialists are all the while attempting to report the vicinity of a bug, then there ought not to be any irregularity at the same time.

- **Portability**

  The framework should to be effectively versatile to another framework. This is obliged when the web server, which s facilitating the framework gets adhered because of a few issues, which requires the framework to be taken to another framework.

- **Scalability**

  The framework should be sufficiently adaptable to include new functionalities at a later stage. There should be a run of the mill channel, which can oblige the new functionalities.

- **Flexibility**

  Flexibility is the capacity of a framework to adjust to changing situations and circumstances, and to adapt to changes to business approaches and rules. An adaptable framework is one that is anything but difficult to reconfigure or adjust because of diverse client and framework prerequisites. The deliberate division of concerns between the trough and motor parts helps adaptability as just a little bit of the framework is influenced when strategies or principles change.

CHAPTER-3

SYSTEM DESIGN & MODELING

3.1. System Architecture

System Architecture design-identifies the overall hypermedia structure for the WebApp. Architecture design is tied to the goals establish for a WebApp, the content to be presented, the users who will visit, and the navigation philosophy that has been established. Content architecture, focuses on the way content objects and structured for presentation and navigation. WebApp architecture, addresses the way the application is structure to manage user interaction, handle internal processing tasks, effect navigation, and present content. WebApp architecture is defined within the context of the development environment in which the application is to be implemented.

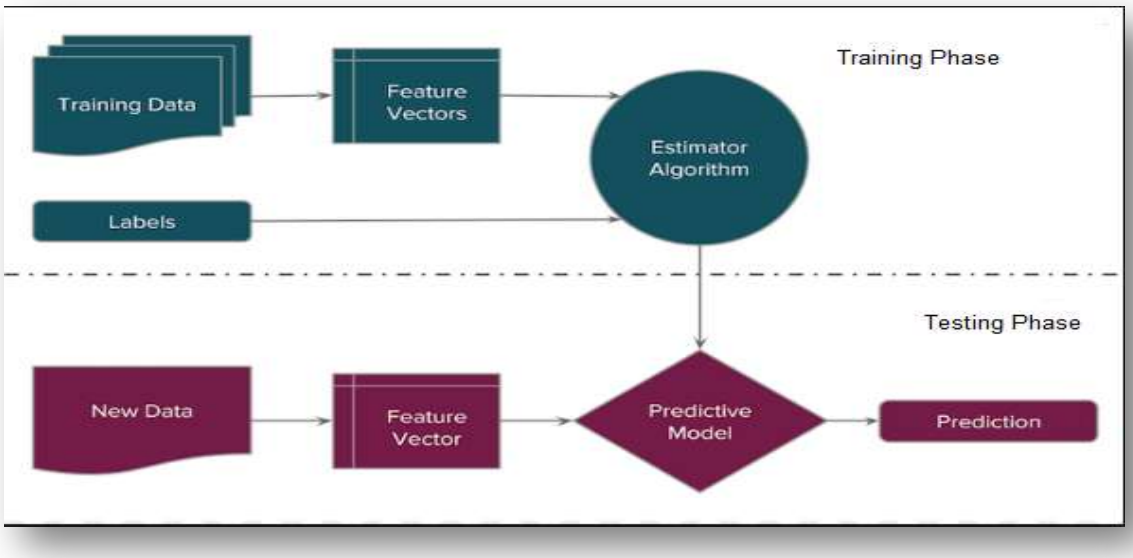


Fig 3.1 System Architecture

3.2 Use case diagrams

A use case is a set of scenarios that describing an interaction between a source and a destination. A use case diagram displays the relationship among actors and use cases. The two main components of a use case diagram are use cases and actors. shows the use case diagram.

3.2.1 Use case diagram User

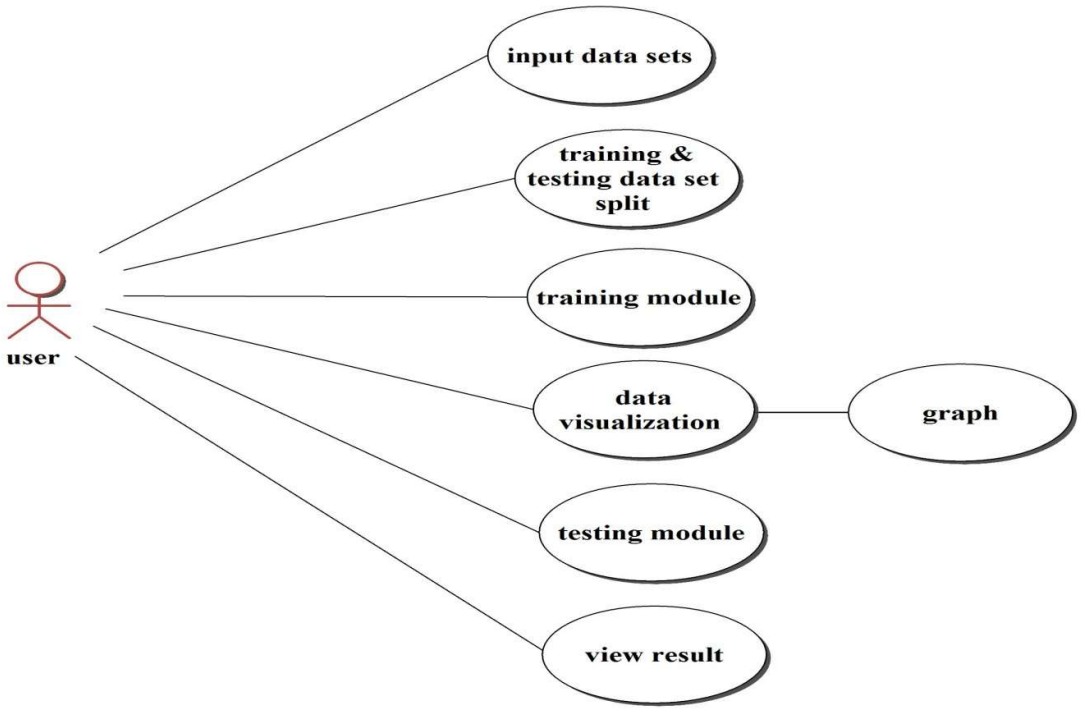


Fig 3.2.1 Use Case Diagram User

3.3 Data flow diagram

A data flow diagram (DFD) is graphic representation of the "flow" of data through an information system. A data flow diagram can also be used for the visualization of data processing (structured design). It is common practice for a designer to draw a context level DFD first which shows the interaction between the system and outside entities. DFD's show the flow of data from external entities into the system, how the data moves from one process to another, as well as its logical storage. There are only four symbols:

1. Squares representing external entities, which are sources and destinations of information entering and leaving the system.
2. Rounded rectangles representing processes, in other methodologies, may be called 'Activities', 'Actions', 'Procedures', 'Subsystems' etc. which take data as input, do processing to it, and output it.
3. Arrows representing the data flows, which can either, be electronic data or physical items. It is impossible for data to flow from data store to data store except via a process, and external entities are not allowed to access data stores directly.

4. The flat three-sided rectangle is representing data stores should both receive information for storing and provide it for further processing.

3.3.1 Level 0 data flow diagram

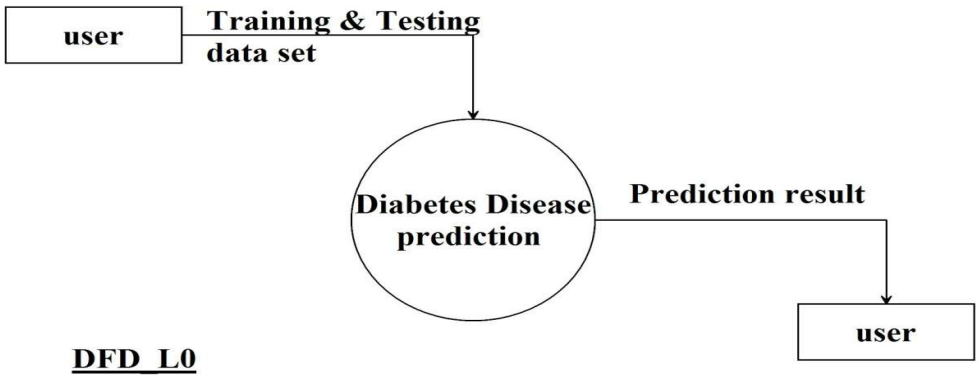


Fig 3.3.1 Level 0Data Flow diagram

3.4.2 Level1 data flow diagram

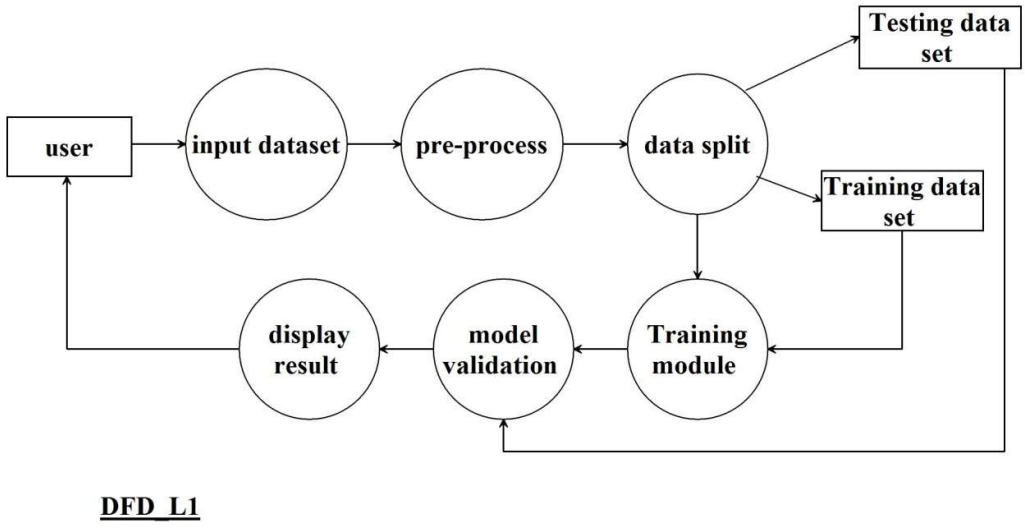


Fig 3.3.2 Level 1 Data Flow Diagram

CHAPTER-4

IMPLEMENTATION TECHNOLOGIES

4.1 Python Introduction:

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.

The Python interpreter and the extensive standard library are freely available in source or binary form for all major platforms from the Python Web site, https://www.python.org/, and may be freely distributed. The same site also contains distributions of and pointers to many free third-party Python modules, programs and tools, and additional documentation.

The Python interpreter is easily extended with new functions and data types implemented in C or C++ (or other languages callable from C). Python is also suitable as an extension language for customizable applications.

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted** − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** − you can sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented** − Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language** − Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

Features:

Python's features include – All these.

- **Easy-to-learn** − Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** − Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** − Python's source code is fairly easy-to-maintain.
- **A broad standard library** − Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** − Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** − you can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** − Python provides interfaces to all major commercial databases.
- **GUI Programming**− Python supports GUI applications that can be created and ported to many system calls, libraries, and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** − Python provides a better structure and support for large programs than shell scripting.

Why python emerging as a leader:

There's battle out there happening in the minds of aspiring data scientists to choose the best data science tool. Though there are quite a number of data science tools that provide the much-needed option, the close combat narrows down between two popular languages – Python and R.

Between the two, Python is emerging as the popular language used more in data science applications.

Take the case of the tech giant Google that has created the deep learning framework called tensor flow – Python is the primary language used for creating this framework. Its footprint has continued to increase in the environment promoted by Netflix. Production engineers at Face book and Khan Academy have for long been using it as a prominent language in their environment.

Python has other advantages that speed up it is upward swing to the top of data science tools. It integrates well with the most cloud as well as platform-as-a-service providers. In supporting multiprocessing for parallel computing, it brings the distinct advantage of ensuring large-scale performance in data science and machine learning. Python can also be extended with modules written in C/C++.

Where Python becomes the perfect-fit

There are tailor-made situations where it is the best data science tool for the job. It is perfect when data analysis tasks involve integration with web apps or when there is a need to incorporate statistical code into the production database. The full-fledged programming nature of Python makes it a perfect fit for implementing algorithms. Its packages rooted for specific data science jobs. Packages like NumPy, SciPy, and pandas produce good results for data analysis jobs. While there is a need for graphics, Python's matplotlib emerges as a good package, and for machine learning tasks, scikit-learn becomes the ideal alternate.

Why is Python preferred over other data science tools?

It is 'Pythonic' when the code is written in a fluent and natural style. Apart from that, it is also known for other features that have captured the imaginations of data science community.

Easy to learn

The most alluring factor of Python is that anyone aspiring to learn this language can learn it easily and quickly. When compared to other data science languages like R, Python promotes a shorter learning curve and scores over others by promoting an easy-to-understand syntax.

Scalability

When compared to other languages like R, Python has established a lead by emerging as a scalable language, and it is faster than other languages like Matlab and Stata. Python's scalability lies in the flexibility that it gives to solve problems, as in the case of YouTube that migrated to Python.

Python has come good for different usages in different industries and for rapid development of applications of all kinds.

Choice of data science libraries

The significant factor giving the push for Python is the variety of data science/data analytics libraries made available for the aspirants. Pandas, Stats Models, NumPy, SciPy, and Scikit-Learn, are some of the libraries well known in the data science community. Python does not stop with that as libraries have been growing over time. What you thought was a constraint a year ago would be addressed well by Python with a robust solution addressing problems of specific nature.

Python community

One of the reasons for the phenomenal rise of Python is attributed to its ecosystem. As Python extends its reach to the data science community, more and more volunteers are creating data science libraries. This, in turn, has led the way for creating the most modern tools and processing in Python.

The widespread and involved community promotes easy access for aspirants who want to find solutions to their coding problems. Whatever queries you need, it is a click or a Google search away. Enthusiasts can also find access to professionals on Code mentor and Stack Overflow to find the right answers for their queries.

Graphics and visualization

Python comes with varied visualization options. Matplotlib provides the solid foundation around which other libraries like Sea born, pandas plotting, and ggplot have been built. The visualization packages help you get a good sense of data, create charts, graphical plot and create web-ready interactive plots.

Why Choose Python?

If you're going to write programs, there are literally dozens of commonly used languages to choose from. Why chooses Python? Here are some of the features that make Python an appealing choice.

Python is Popular

Python has been growing in popularity over the last few years. The 2018 Stack Overflow Developer Survey ranked Python as the 7th most popular and the number one most wanted technology of the year. World-class software development countries around the globe use Python every single day.

According to research by Dice Python is also one of the hottest skills to have and the most popular programming language in the world based on the. Popularity of Programming Language Index

Due to the popularity and widespread use of Python as a programming language, Python developers are sought after and paid well. If you'd like to dig deeper into Python salary statistics and job opportunities, you can do so here.

Many languages are compiled, meaning the source code you create needs to be translated into machine code, the language of your computer's processor, before it can be run. Programs written in an interpreted language are passed straight to an interpreter that runs them directly.

This makes for a quicker development cycle because you just type in your code and run it, without the intermediate compilation step.

One potential downside to interpreted languages is execution speed. Programs that are compiled into the native language of the computer processor tend to run more quickly than interpreted programs. For some applications that are particularly computationally intensive, like graphics processing or intense number crunching, this can be limiting.

In practice, however, for most programs, the difference in execution speed is measured in milliseconds, or seconds at most, and not appreciably noticeable to a human user. The expediency of coding in an interpreted language is typically worth it for most applications.

Python is Free

The Python interpreter is developed under an OSI-approved open-source license, making it free to install, use, and distribute, even for commercial purposes. A version of the interpreter is available for virtually any platform there is, including all Flavors of Unix, Windows, macOS, smartphones and tablets, and probably anything else you ever heard of. A version even exists for the half dozen people remaining who use OS/2.

Python is Portable

Because Python code is interpreted and not compiled into native machine instructions, code written for one platform will work on any other platform that has the Python interpreter installed. (This is true of any interpreted language, not just Python.)

Python is Simple

As programming languages go, Python is relatively uncluttered, and the developers have deliberately kept it that way. A rough estimate of the complexity of a language can be gleaned from the number of keywords or reserved words in the language. These are words that are reserved for special meaning by the compiler or interpreter because they designate specific built-in functionality of the language. Python 3 has 33 keywords, and Python 2 has 31. By contrast, C++ has 62, Java has 53, and Visual Basic has more than 120, though these latter examples probably vary somewhat by implementation or dialect. Python code has a simple and clean structure that is easy to learn and easy to read. In fact, as you will see, the language definition enforces code structure that is easy to read.

But It's Not That Simple

For all its syntactical simplicity, Python supports most constructs that would be expected in a very high-level language, including complex dynamic data types, structured and functional programming, and object-oriented programming. Additionally, a very extensive library of classes and functions is available that provides capability well beyond what is built into the language, such as database manipulation or GUI programming.

Python accomplishes what many programming languages don't: the language itself is simply designed, but it is very versatile in terms of what you can accomplish with it.

Is Python 'the' tool for machine learning?

When it comes to data science, machine learning is one of the significant elements used to maximize value from data. With Python as the data science tool, exploring the basics of machine learning becomes easy and effective. In a nutshell, machine learning is more about statistics, mathematical optimization, and probability. It has become the most preferred machine learning tool in the way it allows aspirants to 'do math' easily.

Name any math function, and you have a Python package meeting the requirement. There is NumPy for numerical linear algebra, CVXOPT for convex optimization, SciPy for general scientific computing, SymPy for symbolic algebra, PYMC3, and Stats model for statistical modeling.

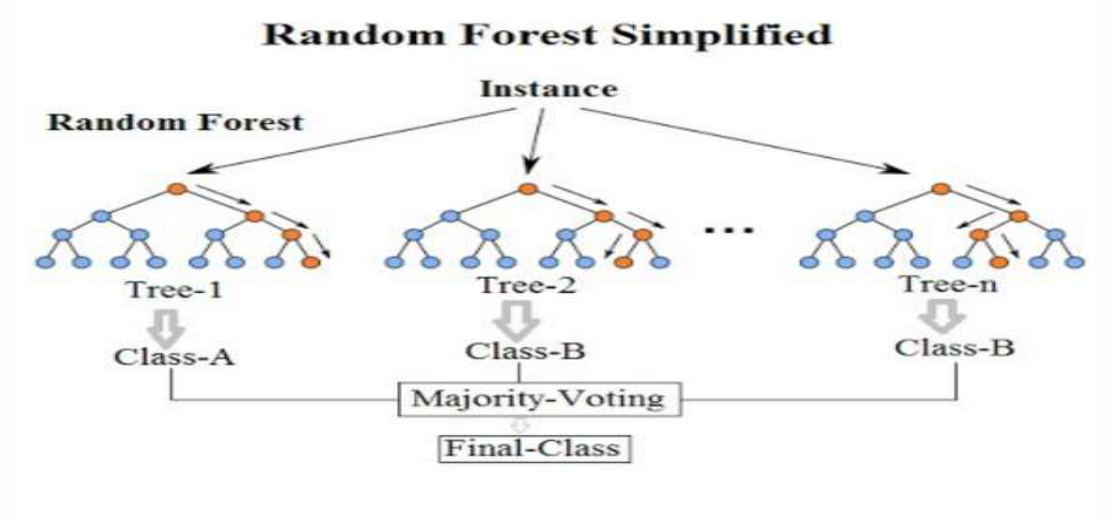
With the grip on the basics of machine learning algorithm including logistic regression and linear regression, it makes it easy to implement machine learning systems for predictions by way of its scikit-learn library. It is easy to customize for neutral networks and deep learning with libraries including Keras, Theano, and TensorFlow.

Data science landscape is changing rapidly, and tools used for extracting value from data science have also grown in numbers. The two most popular languages that fight for the top spot are R and Python. Both are revered by enthusiasts, and both come with their strengths and weaknesses. But with the tech giants like Google showing the way to use Python and with the learning curve made short and easy, it inches ahead to become the most popular language in the data science world.

4.2 Algorithms used:

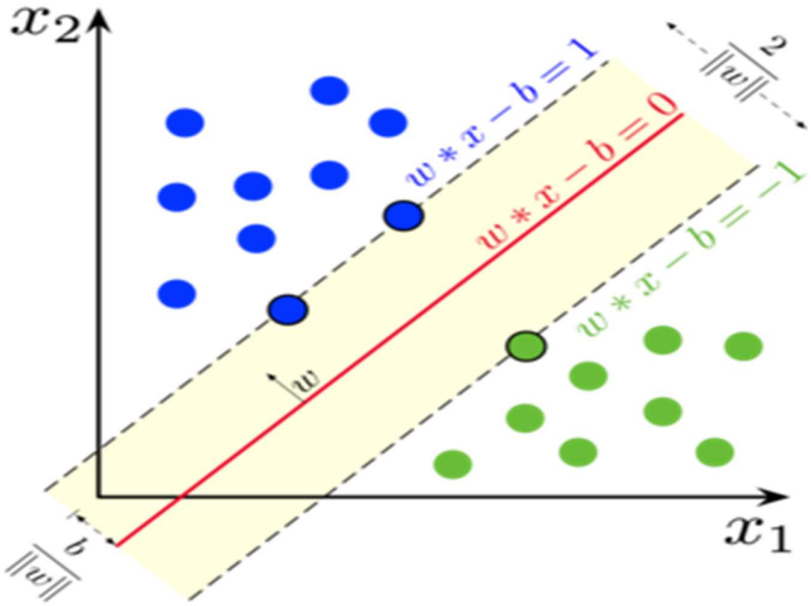
Random Forest Algorithm

Random forests or random decision forests are an ensemble learning method for classification, regression and other tasks that operate by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes or mean prediction of the individual trees. Random Forest Classifier being ensemble algorithm tends to give more accurate result. This is because it works on principle, Number of weak estimators when combined forms strong estimator.



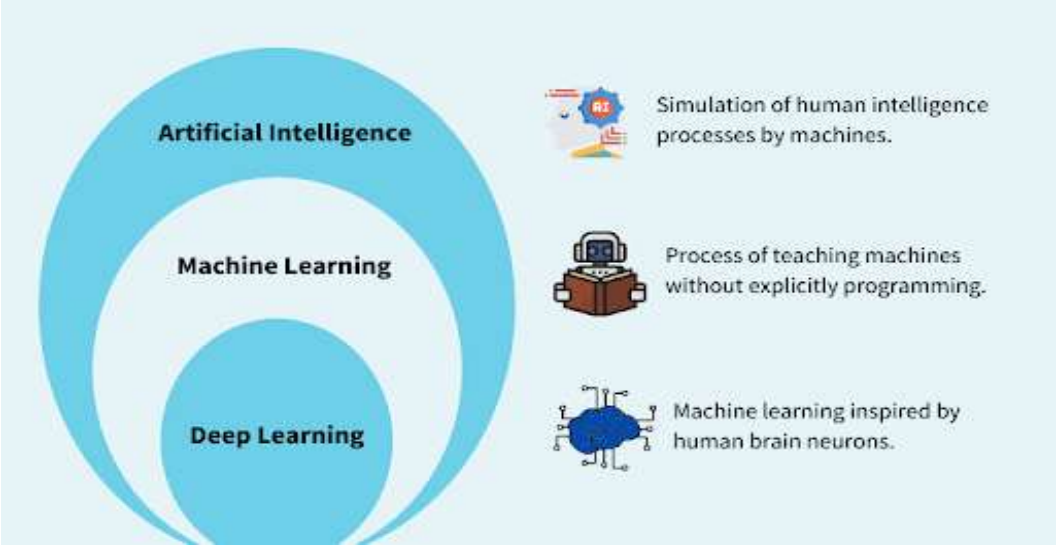
Support Vector Machine (SVM)

A support vector machine (SVM) is machine learning algorithm that analyzes data for classification and regression analysis. SVM is a supervised learning method that looks at data and sorts it into one of two categories. ... A support vector machine is also known as a support vector network (SVN).



4.3 Artificial intelligence

Artificial Intelligence encompasses a very broad scope. You could even consider something like Dijkstra's shortest path algorithm as Artificial Intelligence. However, two categories of AI are frequently mixed up: Machine Learning and Deep Learning. Both refer to statistical modeling of data to extract useful information or make predictions. In this article, we will list the reasons why these two statistical modeling techniques are not the same and help you further frame your understanding of these data modeling paradigms.



4.4 Machine learning

Machine Learning is a method of statistical learning where each instance in a dataset is described by a set of features or attributes. In contrast, the term "Deep Learning" is a method of statistical learning that extracts features or attributes from raw data. Deep Learning does this by utilizing neural networks with many hidden layers, big data, and powerful computational resources. The terms seem somewhat interchangeable, however, with Deep Learning method, The algorithm constructs representations of the data automatically. In contrast, data representations are hard-coded as a set of features in machine learning algorithms, requiring further processes such as feature selection and extraction, (such as PCA).

Both terms are in dramatic contrast with another class of classical artificial intelligence algorithms known as Rule-Based Systems where each decision is manually programmed in such a way that it resembles a statistical model.

In Machine Learning and Deep Learning, there are many different models that fall into two different categories, supervised and unsupervised. In unsupervised learning, algorithms such as k-Means, hierarchical clustering, and Gaussian mixture models attempt to learn meaningful structures in the data. Supervised learning involves an output label associated with each instance in the dataset. This output can be discrete/categorical or real-valued. Regression models estimate real-valued outputs, whereas classification models estimate discrete-valued outputs. Simple binary classification models have just two output labels, 1 (positive) and 0 (negative). Some popular supervised learning algorithms that are considered Machine Learning: are linear regression, logistic regression, decision trees, support vector machines, and neural networks, as well as non-parametric models such as k-Nearest Neighbors.

Supervised Machine Learning

Most of the practical machine learning uses supervised learning.

Supervised learning is where you have input variables (x) and an output variable (Y) and you use an algorithm to learn the mapping function from the input to the output.

$Y = f(X)$

The goal is to approximate the mapping function so well that when you have new input data (x) that you can predict the output variables (Y) for that data.

It is called supervised learning because the process of an algorithm learning from the training dataset can be thought of as a teacher supervising the learning process. We know the correct answers, the algorithm iteratively makes predictions on the training data and is corrected by the teacher. Learning stops when the algorithm achieves an acceptable level of performance.

Supervised learning problems can be further grouped into regression and classification problems.

Classification: A classification problem is when the output variable is a category, such as "red" or "blue" or "disease" and "no disease".

Regression: A regression problem is when the output variable is a real value, such as "dollars" or "weight".
Some common types of problems built on top of classification and regression include recommendation and time series prediction respectively.

Some popular examples of supervised machine learning algorithms are:

- Linear regression for regression problems.
- Random forest for classification and regression problems.
- Support vector machines for classification problems.

Unsupervised Machine Learning

Unsupervised learning is where you only have input data (X) and no corresponding output variables. The goal for unsupervised learning is to model the underlying structure or distribution in the data in

order to learn more about the data. These are called unsupervised learning because unlike supervised learning above there is no correct answers and there is no teacher. Algorithms are left to their own devises to discover and present the interesting structure in the data. Unsupervised learning problems can be further grouped into clustering and association problems.

- **Clustering**: A clustering problem is where you want to discover the inherent groupings in the data, such as grouping customers by purchasing behaviour.
- **Association**: An association rule learning problem is where you want to discover rules that describe large portions of your data, such as people that buy X also tend to buy Y.
  Some popular examples of unsupervised learning algorithms are:

- k-means for clustering problems.

- A priori algorithm for association rule learning problems.

Semi-Supervised Machine Learning

Problems where you have a large amount of input data (X) and only some of the data is labelled (Y) are called semi-supervised learning problems.

These problems sit in between both supervised and unsupervised learning.

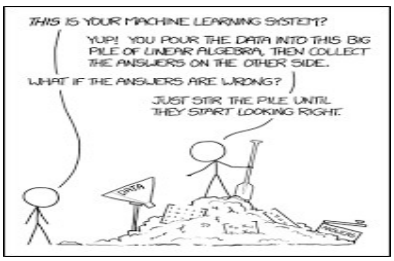
A good example is a photo archive where only some of the images are labelled (e.g., dog, cat, person) and the majority are unlabeled.to domain experts. Whereas unlabelled data is cheap and easy to collect and store. You can use unsupervised learning techniques to discover and learn the structure in the input variables. You can also use supervised learning techniques to make best guess predictions for the unlabelled data, feed that data back into the supervised learning algorithm as training data and use the model to make predictions on new unseen data.

Data Size:

Both Machine Learning and Deep Learning can handle massive dataset sizes, however, machine learning methods make much more sense with small datasets. For example, if you only have 100 data points, decision trees, k-nearest neighbours, and other machine learning models will be much more valuable to you than fitting a deep neural network on the data. This is due to the next topic of difference, Interpretability

Interpretability:

Example for how interpretability works in ML & DL:



A lot of the criticism of deep learning methods and machine learning algorithms such as Support Vector Machine or (maybe, because you can at least visualize the constituent probabilities making up the output), Naive Bayes, are due to their difficulty to interpret. For example, when a Convolutional Neural Network outputs 'cat' in a dog vs. cat problem, nobody seems to know why it did that. In contrast, when you are modeling data such as an electronic health record or bank loan dataset with a machine learning technique, it is much easier to understand the reasoning for the model's prediction.

One of the best examples of interpretability is decision trees where you follow logical tests down nodes of the tree until you reach a decision. Another machine learning algorithm with high interpretability is k-Nearest Neighbours. This is not a parametric learning algorithm but still falls under the category of machine learning algorithms. It is very interpretability because you easily reason about the similar instances for yourself.

4.5 SDLC Project Methodology

It will cover the details explanation of methodology that is being used to make this project complete and working well. Many methodologies or findings from this field mainly generated into journal for others to take advantages and improve as upcoming studies. The method is use to achieve the objective of the project that will accomplish a perfect result. In order to evaluate this project, the methodology based on System Development Life Cycle (SDLC), generally three major steps, which is planning, implementing and analysis.

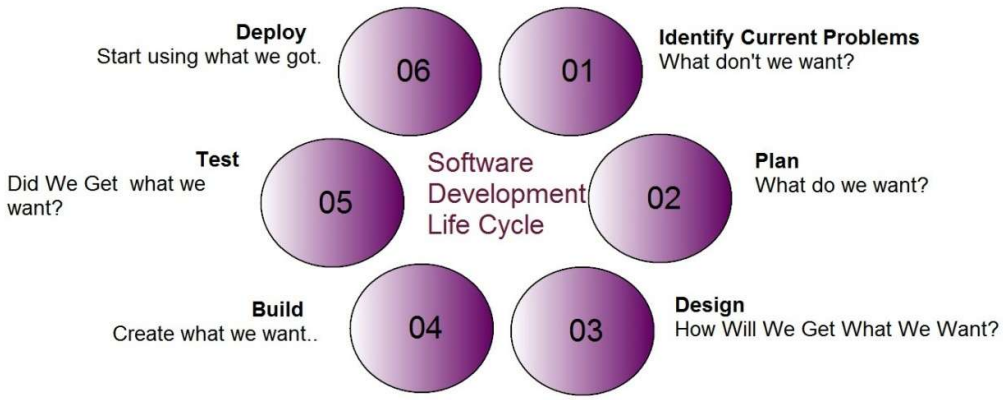


Fig 1: Software Development Life Cycle

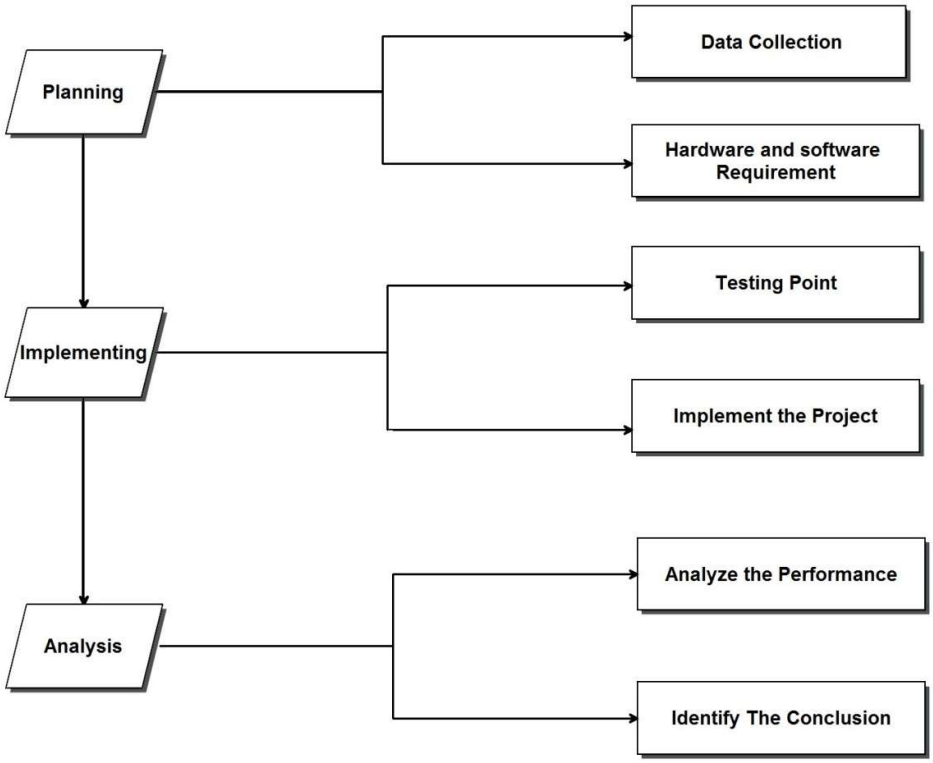


Fig 2: Steps of Methodology

Planning:

To identify all the information and requirement such as hardware and software, planning must be done in the proper manner. The planning phase has two main elements namely data collection and the requirements of hardware and software.

Data collection:

Machine learning needs two things to work, data (lots of it) and models. When acquiring the data, be sure to have enough features (aspect of data that can help for a prediction, like the surface of the house to predict its price) populated to train correctly your learning model. In general, the more data you have the better so make to come with enough rows.

The primary data collected from the online sources remains in the raw form of statements, digits, and qualitative terms. The raw data contains error, omissions and inconsistencies. It requires corrections after careful scrutinizing the completed questionnaires. The following steps are involved in the processing of primary data. A huge volume of raw data collected through field survey needs to be grouped for similar details of individual responses.

Data Preprocessing is a technique that is used to convert the raw data into a clean data set. In other words, whenever the data is gathered from different sources it is collected in raw format which is not feasible for the analysis.

Therefore, certain steps are executed to convert the data into a small clean data set. This technique is performed before the execution of Iterative Analysis. The set of steps is known as Data Preprocessing. It includes -

- Data Cleaning
- Data Integration
- Data Transformation
- Data Reduction

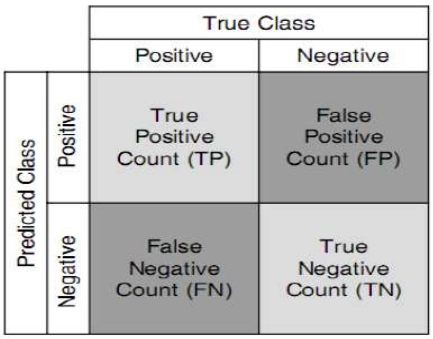
Data Preprocessing is necessary because of the presence of unformatted real-world data. Mostly real-world data is composed of -

- **Inaccurate data (missing data) -** There are many reasons for missing data such as data is not continuously collected, a mistake in data entry, technical problems with biometrics and much more.
- **The presence of noisy data (erroneous data and outliers) -** The reasons for the existence of noisy data could be a technological problem of gadget that gathers data, a human mistake during data entry and much more.
- **Inconsistent data -** The presence of inconsistencies are due to the reasons such that existence of duplication within data, human data entry, containing mistakes in codes or names, i.e., violation of data constraints and much more.

Analysis

In this final phase, we will test our classification model on our prepared image dataset and also measure the performance on our dataset. To evaluate the performance of our created classification and make it comparable to current approaches, we use accuracy to measure the effectiveness of classifiers.

After model building, knowing the power of model prediction on a new instance, is very important issue. Once a predictive model is developed using the historical data, one would be curious as to how the model will perform on the data that it has not seen during the model building process. One might even try multiple model types for the same prediction problem, and then, would like to know which model is the one to use for the real-world decision-making situation, simply by comparing them on their prediction performance (e.g., accuracy). To measure the performance of a predictor, there are commonly used performance metrics, such as accuracy, recall etc. First, the most used performance metrics will be described, and then some famous estimation methodologies are explained and compared to each other. "Performance Metrics for Predictive Modeling In classification problems, the primary source of performance measurements is a coincidence matrix (**classification matrix or a contingency table**)". Above figure shows a coincidence matrix for a two-class classification problem. The equations of the most used metrics that can be calculated from the coincidence matrix.



$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{True Negative Rate} = \frac{TN}{TN + FP}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

Figure 3: confusion matrix and formulae

As being seen in above figure, the numbers along the diagonal from upper-left to lower-right represent the correct decisions made, and the numbers outside this diagonal represent the errors. "The true positive rate (also called hit rate or recall) of a classifier is estimated by dividing the correctly classified positives (the true positive count) by the total positive count. The false positive rate (also called a false alarm rate) of the classifier is estimated by dividing the incorrectly classified negatives (the false negative count) by the total negatives. The overall accuracy of a classifier is estimated by dividing the total correctly classified positives and negatives by the total number of samples.

The architecture of a ConvNet is analogous to that of the connectivity pattern of Neurons in the Human Brain and was inspired by the organization of the Visual Cortex. Individual neurons respond to stimuli only in a restricted region of the visual field known as the Receptive Field. A collection of such fields overlaps to cover the entire visual area.

Flexibility

Sometimes you just do not want to use what is already there but you want to define something of your own (for example a cost function, a metric, a layer, etc.).
Although Keras 2 has been designed in such a way that you can implement almost everything you want but we all know that low-level libraries provide more flexibility. Same is the case with TF. *You can tweak* TF much more as compared to Keras.

Functionality

Although Keras provides all the general-purpose functionalities for building Deep learning models, it does not provide as much as TF. TensorFlow offers more advanced operations as compared to Keras.

This comes very handy if you are doing research or developing some special kind of deep learning models. Some examples regarding high level operations are**:**

Threading and Queues

Queues are a powerful mechanism for computing tensors asynchronously in a graph. Similarly, you can execute multiple threads for the same Session for parallel computations and hence speed up your operations.

Debugger

Another extra power of TF. With TensorFlow, you get a specialized debugger. It provides visibility into the internal structure and states of running TensorFlow graphs. Insights from debugger can be used to facilitate debugging of various types of bugs during both training and inference.

Control

The more control you have over your network, more better understanding you have of what is going on with your network.
With TF, you get such a control over your network. You can control whatever you want in your network. Operations on weights or gradients can be done like a charm in TF.

NumPy

NumPy, which stands for Numerical Python, is a library consisting of multidimensional array objects and a collection of routines for processing those arrays. Using NumPy, mathematical and logical operations on arrays can be performed. This tutorial explains the basics of NumPy such as its architecture and environment. It also discusses the various array functions, types of indexing, etc. An introduction to Matplotlib is also provided. All this is explained with the help of examples for better understanding.

NumPy is a Python package. It stands for 'Numerical Python'. It is a library consisting of multidimensional array objects and a collection of routines for processing of array.

Numeric, the ancestor of NumPy, was developed by Jim Hugunin. Another package Numarray was also developed, having some additional functionality. In 2005, Travis Oliphant created NumPy package by incorporating the features of Numarray into Numeric package. There are many contributors to this open-source project.

Operations using NumPy

Using NumPy, a developer can perform the following operations −

- Mathematical and logical operations on arrays.
- Fourier transforms and routines for shape manipulation.
- Operations related to linear algebra. NumPy has in-built functions for linear algebra and random number generation.

sNumPy A Replacement for Mat Lab

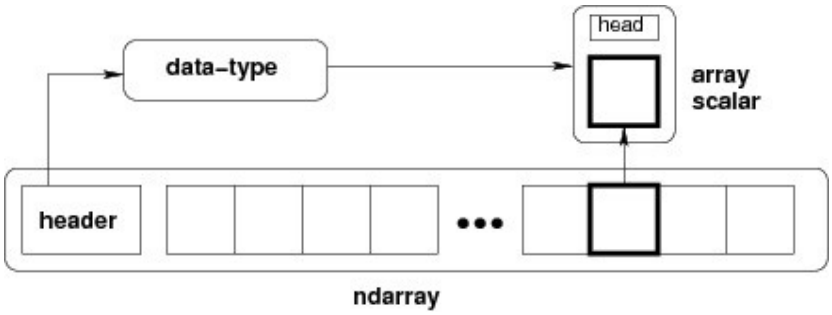
NumPy is often used along with packages like **SciPy** (Scientific Python) and **Matplotlib** (plotting library). This combination is widely used as a replacement for MatLab, a popular platform for technical computing. However, Python alternative to MatLab is now seen as a more modern and complete programming language.

It is open source, which is an added advantage of NumPy.

The most important object defined in NumPy is an N-dimensional array type called **ndarray**. It describes the collection of items of the same type. Items in the collection can be accessed using a zero-based index.

Every item in an ndarray takes the same size of block in the memory. Each element in ndarray is an object of data-type object (called **dtype**).

Any item extracted from ndarray object (by slicing) is represented by a Python object of one of array scalar types. The following diagram shows a relationship between ndarray, data type object (dtype) and array scalar type −



An instance of ndarray class can be constructed by different array creation routines described later in the tutorial. The basic ndarray is created using an array function in NumPy as follows −

```
numpy.array
```

It creates an ndarray from any object exposing array interface, or from any method that returns an array.

The ndarray objects can be saved to and loaded from the disk files. The IO functions available are −

- **load()** and **save()** functions handle /numPy binary files (with **npy**extension)
- **loadtxt()** and **savetxt()** functions handle normal text files

NumPy introduces a simple file format for ndarray objects. This **.npy** file stores data, shape, dtype and other information required to reconstruct the ndarray in a disk file such that the array is correctly retrieved even if the file is on another machine with different architecture.

numpy. save ()

The **numpy. save ()** file stores the input array in a disk file with **npy**extension.

```
import numpy as np

a = np. array ([1,2,3,4,5])

np.save('outfile',a)
```

To reconstruct array from **outfile.npy**, use **load()** function.

```
import numpy as np
```

Pandas:

pandas are a Python package providing fast, flexible, and expressive data structures designed to make working with structured (tabular, multidimensional, potentially heterogeneous) and time series data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, **real world** data analysis in Python. Additionally, it has the broader goal of becoming **the most powerful and flexible open-source data analysis / manipulation tool available in any language**. It is already well on its way toward this goal.

pandas are well suited for many kinds of data:

- Tabular data with heterogeneously-typed columns, as in an SQL table or Excel spreadsheet
- Ordered and unordered (not necessarily fixed-frequency) time series data.
- Arbitrary matrix data (homogeneously typed or heterogeneous) with row and column labels

- Any other form of observational / statistical data sets. The data need not be labelled at all to be placed into a panda's data structure

The two primary data structures of pandas, Series (1-dimensional) and Data Frame (2-dimensional), handle the vast majority of typical use cases in finance, statistics, social science, and many areas of engineering. For R users, Data Frame provides everything that R's data. Frame provides and much more. Pandas is built on top of NumPy and is intended to integrate well within a scientific computing environment with many other 3rd party libraries.

Here are just a few of the things that pandas do well:

- Easy handling of **missing data** (represented as NaN) in floating point as well as non-floating-point data
- Size mutability: columns can be **inserted and deleted** from DataFrame and higher dimensional objects
- Automatic and explicit **data alignment**: objects can be explicitly aligned to a set of labels, or the user can simply ignore the labels and let *Series*, *DataFrame*, etc. automatically align the data for you in computations
- Powerful, flexible **group by** functionality to perform split-apply-combine operations on data sets, for both aggregating and transforming data
- Make it **easy to convert** ragged, differently-indexed data in other Python and NumPy data structures into DataFrame objects
- Intelligent label-based **slicing**, **fancy indexing**, and **subsetting** of large data sets
- Intuitive **merging** and **joining** data sets
- Flexible **reshaping** and pivoting of data sets
- **Hierarchical** labeling of axes (possible to have multiple labels per tick)
- Robust IO tools for loading data from **flat files** (CSV and delimited), Excel files, databases, and saving / loading data from the ultrafast **HDF5 format**
- **Time series**-specific functionality: date range generation and frequency conversion, moving window statistics, moving window linear regressions, date shifting and lagging, etc.
- Many of these principles are here to address the shortcomings frequently experienced using other languages / scientific research environments. For data scientists, working with data is typically divided into multiple stages: munging and cleaning data, analyzing / modeling it, then organizing the results of the analysis into a form suitable for plotting or tabular display. pandas are the ideal tool for all these tasks.

Matplotlib

Matplotlib is an amazing visualization library in Python for 2D plots of arrays. Matplotlib is a multi-platform data visualization library built on NumPy arrays and designed to work with the broader SciPy stack. It was introduced by John Hunter in the year 2002.One of the greatest benefits of visualization is that it allows us visual access to huge amounts of data in easily digestible visuals. Matplotlib consists of several plots like line, bar, scatter, histogram etc.

Uses of matplotlib

- Matplotlib is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-**oriented** API for **embedding**plots into applications using general-purpose GUI toolkits like Tkinter, wxPython, Qt, or GTK+.
- MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object-oriented interface or via a set of functions familiar to MATLAB users.
- Matplotlib is a Python package for 2D plotting that generates production-quality graphs. It supports interactive and non-interactive plotting, and can save images in several output formats (PNG, PS, and others). It can use multiple window toolkits (GTK+, wxWidgets, Qt, and so on) and it provides a wide variety of plot types (lines, bars, pie charts, histograms, and many more).
- In addition to this, it is highly customizable, flexible, and easy to use. The dual nature of Matplotlib allows it to be used in both interactive and non-interactive scripts. It can be used in scripts without a graphical display, embedded in graphical applications, or on web pages. It can also be used interactively with the Python interpreter or IPython.

Merits of Matplotlib

- The idea behind Matplotlib can be summed up in the following motto as quoted by John Hunter, the creator and project leader of Matplotlib:
- “Matplotlib tries to make easy things easy and hard things possible”.
- Matplotlib was born in the scientific area of computing, where gnuplot and MATLAB were (and still are) used a lot.

- Matplotlib was modeled on MATLAB, because graphing was something that MATLAB did very well. The high degree of compatibility between them made many people move from MATLAB to Matplotlib, as they felt like home while working with Matplotlib.
- But what are the points that built the success of Matplotlib? Let's look at some of them:
- **It uses Python:** Python is a very interesting language for scientific purposes (it is interpreted, high-level, easy to learn, easily extensible, and has a powerful standard library) and is now used by major institutions such as NASA, JPL, Google, DreamWorks, Disney, and many more.
- **It is open source, so no license to pay:** This makes it very appealing for professors and students, who often have a low budget.
- **It is a real programming language:** The MATLAB language (while being Turing-complete) lacks many of the features of a general-purpose language like Python.
- **It is much more complete:** Python has a lot of external modules that will help us perform all the functions we need to. So, it is the perfect tool to acquire data, elaborate the data, and then plot the data.
- **It is very customizable and extensible:** Matplotlib can fit every use case because it has a lot of graph types, features, and configuration options.
- **It is integrated with LaTeX markup:** This is useful when writing scientific papers.
- **It is cross-platform and portable:** Matplotlib can run on Linux, Windows, Mac OS X, and Sun Solaris (and Python can run on almost every architecture available).
- The aim of Matplotlib is to generate graphs. So, we need a way to actual view these images or even to save them to files. We are going to look at the various output formats available in Matplotlib and the graphical user interfaces (GUIs) supported by the library.
- A backend that displays the image on screen is called a user interface backend.
- The backend is that part of Matplotlib that works behind the scenes and allows the software to target several different output formats and GUI libraries (for screen visualization).
- In order to be even more flexible, Matplotlib introduces the following two layers structured (only for GUI output):
- **The renderer:** This does the drawing

- **The canvas:** This is the destination of the figure.
- The standard renderer is the Anti-Grain Geometry (AGG) library, a high-performance rendering engine which can create images of publication level quality, with anti-aliasing, and sub pixel accuracy. AGG is responsible for the beautiful appearance of Matplotlib graphs.
- The canvas is provided with the GUI libraries, and any of them can use the AGG rendering, along with the support for other rendering engines (for example, GTK+).

S K Learn scikit-learn

Introduction

scikit-learn is a library, i.e., a collection of classes and functions that users import into Python programs. Using scikit-learn therefore requires basic Python programming knowledge. No command-line interface, let alone a graphical user interface, is offered for non-programmer users

Scikit-learn is the most useful library for machine learning in Python. It is on NumPy, SciPy and matplotlib, this library contains a lot of efficient tools for machine learning and statistical modeling including classification, regression, clustering, and dimensionality reduction.

Scikit-learn provide a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Scikit-learn is an opensource Python library that has powerful tools for data analysis and data mining. It's available under the BSD license and is built on the following machine learning libraries: NumPy, a library for manipulating multi-dimensional arrays and matrices.

Non-proliferation of classes: Learning algorithms are the only objects to be represented using custom classes. Datasets are represented as NumPy arrays or SciPy sparse matrices. Hyper-parameter names and values are represented as standard Python strings or numbers whenever possible. This keeps scikit learn easy to use and easy to combine with other libraries.

Composition: Many machine learning tasks are expressible as sequences or combinations of transformations to data. Some learning algorithms are also naturally viewed as meta-algorithms parameterized on other algorithms. Whenever feasible, such algorithms are implemented and composed from existing building blocks. Sensible defaults. Whenever an operation requires a user-

defined parameter, an appropriate default value is defined by the library. The default value should cause the operation to be performed in a sensible way (giving a baseline solution for the task at hand).

Components of scikit-learn:

Scikit-learn comes loaded with a lot of features. Here are a few of them to help you understand the spread:

- **Supervised learning algorithms:** Think of any supervised learning algorithm you might have heard about and there is a very high chance that it is part of scikit-learn. Starting from Generalized linear models (e.g Linear Regression), Support Vector Machines (SVM), Decision Trees to Bayesian methods – all of them are **part of** scikit-learn toolbox. The spread of algorithms is one of the big reasons for high usage of scikit-learn. I started using scikit to solve supervised learning problems and would recommend that to people new to scikit / machine learning as well.

Cross-validation: There are various methods to check the accuracy of supervised models on unseen data.

Unsupervised learning algorithms: Again, there is a large spread of algorithms in the offering – starting from clustering, factor analysis, principal component analysis to unsupervised neural networks.

Various toy datasets: This came in handy while learning scikit-learn. I had learnt SAS using various academic datasets (e.g. IRIS dataset, Boston House prices dataset). Having them handy while learning a new library helped a lot.**Feature extraction:** Useful for extracting features from images and text (e.g. Bag of words)

CHAPTER-5

SYSTEM REQUIREMENT SPECIFICATION

A Software Requirement Specification (SRS) is basically an organization's understanding of a customer or potential client's system requirements and dependencies at a particular point prior to any actual design or development work. The information gathered during the analysis is translated into a document that defines a set of requirements. It gives the brief description of the services that the system should provide and also the constraints under which, the system should operate. Generally, the SRS is a document that completely describes what the proposed software should do without describing how the software will do it. It's a two-way insurance policy that assures that both the client and the organization understand the other's requirements from that perspective at a given point in time.

The SRS document itself states in precise and explicit language those functions and capabilities a software system must provide, as well as states any required constraints by which the system must abide. The SRS also functions as a blueprint for completing a project with as little cost growth as possible. The SRS is often referred to as the "parent" document because all subsequent project management documents, such as design specifications, statements of work, software architecture specifications, testing and validation plans, and documentation plans, are related to it. Requirement is a condition or capability to which the system must conform. Requirement Management is a systematic approach towards eliciting, organizing and documenting the requirements of the system clearly along with the applicable attributes. The elusive difficulties of Requirements are not always obvious and can come from any number of sources.

Hardware Requirements

- System Processor : Core i3 / i5
- Hard Disk : 500 GB.
- Ram : 4 GB.

✓ *Any desktop / Laptop system with above configuration or higher level.*

Software Requirements

- Operating system : Windows 8 / 10 OS
- Programming Language : Python
- Framework : Anaconda
- IDE : Jupyter Notebook
- DL Libraries : Numpy, Pandas

CHAPTER -6

TESTING AND IMPLEMENTATION

6.1 Software testing introduction

Software testing is a process used to help identify the correctness, completeness and quality of developed computer software. Software testing is the process used to measure the quality of developed software. Testing is the process of executing a program with the intent of finding errors. Software testing is often referred to as verification & validation

6.2 Explanation for SDLC & STLC

SDLC: The software development life cycle (SDLC) is a conceptual model used in project management that describes the stages involved in an information system development project, from an initial feasibility study through maintenance of the completed application.

6.3 PHASES OF SOFTWARE DEVELOPMENT

- Requirement Analysis
- Software design
- Development or Coding
- Testing
- Maintenance

6.3.1 Requirement analysis

The requirements of a desired software product are extracted. Based the business scenario the SRS (Software Requirement Specification) document is prepared in this phase.

6.3.2 Design

Plans are laid out concerning the physical construction, hardware, operating systems, programming, communications, and security issues for the software. Design phase is concerned with making sure the software system will meet the requirements of the product.

There are 2 stages in design,

HLD – High Level Design

LLD – Low Level Design

HLD – gives the architecture of the software product to be developed and is done by architects and senior developers.

LLD – done by senior developers. It describes how each and every feature in the product should work and how every component should work. Here, only the design will be there and not the code.

6.3.3 Testing

Testing is evaluating the software to check for the user requirements. Here the software is evaluated with intent of finding defects.

6.3.4 Maintenance

Once the new system is up and running for a while, it should be exhaustively evaluated. Maintenance must be always kept up rigorously. Users of the system should be kept up-to-date concerning the latest modifications and procedures

6.4 SDLC Models

6.4.1 Water fall model

It will be executing one by one of the SDLC process. The design Starts after completing the requirements analysis coding begins after design. It is a traditional model It is a sequential design process, often used in SDLC, in which the progress is seen as flowing steadily downwards (like a waterfall), through the different phases.

6.4.2 Prototype model

Developed from the sample after getting good feedback from the customer. This is the Valuable mechanism for gaining better understanding of the customer needs

6.4.3 Rapid application development model (RAD):

This mechanism will develop from already existing one. If The New requirement is matching in already existing requirement, will develop from that**.**

6.4.4 Spiral model

This mechanism is updating the application version by version. All the SDLC process will update version by version**.**

6.4.5 V-MODELV:

V model is a process where the development and testing phases can do parallel. For every development phase there is a testing phase. Development phases are called as verification whereas testing phases are called as validation

6.5 STLC (Software Testing Life Cycle): Testing itself has many phases i.e., is called as STLC. STLC is part of SDLC

Test Plan

Test Development

Test Execution

Analyze Results

Defect Tracking

Summaries Report

6.5.1. TEST PLAN

It is a document which describes the testing environment, purpose, scope, objectives, test strategy, schedules, mile stones, testing tool, roles and responsibilities, risks, training, staffing and who is going to test the application, what type of tests should be performed and how it will track the defects.

6.5.2. TEST DEVELOPMENT

Preparing test cases, test data, preparing test procedure, preparing test scenario, Writing test script

6.5.3 TEST EXECUTION

In this phase we execute the documents those are prepared in test development phase

6.5.4 ANALYZE RESULT:

Once executed documents will get results either pass or fail. we need to analyze the results during this phase.

6.5.5 DEFECT TRACKING:

Whenever we get defect on the application, we need to prepare the bug report file and forwards to Test Team Lead and Dev Team. The Dev Team will fix the bug. Again, we have to test the application. This cycle repeats till we get the software without defects.

6.6 TYPES OF TESTING:

- White Box Testing
- Black Box Testing
- Grey box testing

6.6.1 WHITEBOXTESTING

White box testing as the name suggests gives the internal view of the software. This type of testing is also known as structural testing or glass box testing as well, as the interest lies in what lies inside the box.

6.6.2 BLACK BOX TESTING

It is also called as behavioural testing. It focuses on the functional requirements of the software. Testing either functional or non-functional without reference to the internal structure of the component or system is called black box testing.

6.6.3 GREY BOX TESTING

Grey box testing is the combination n of black box and white box testing. Intention of this testing is to find out defects related to bad design or bad implementation of the system.

6.7 LEVEL OF TESTING USED IN PROJECT

6.7.1 Unit testing

Initialization testing is the first level of dynamic testing and is first the responsibility of developers and then that of the test engineers. Unit testing is performed after the expected test results are met or differences are explainable/acceptable**.**

6.7.2 Integration testing

All module which makes application are tested. Integration testing is to make sure that the interaction of two or more components produces results that satisfy functional requirement**.**

6.7.3 System testing

To test the complete system in terms of functionality and non-functionality. It is black box testing, performed by the Test Team, and at the start of the system testing the complete system is configured in a controlled environment.

6.7.4 Functional testing

The outgoing links from all the pages from specific domain under test. Test all internal links. Test links jumping on the same pages. Check for the default values of fields. Wrong inputs to the fields in the forms.

6.7.5 Alpha testing

Alpha testing is final testing before the software is released to the general public. This testing is conducted at the developer site and in a controlled environment by the end user of the software.

6.7.6 Beta testing

The beta test is conducted at one or more customer sites by the end user of the software. The beta test is conducted at one or more customer sites by the end user of the software.

6.8-unit testing cases

Initialization testing is the first level of dynamic testing and is first the responsibility of developers and then that of the test engineers. Unit testing is performed after the expected test results are met or differences are explainable/acceptable**.**

TESTCASE

Id	Test Case Title	Test Input	Result	Remarks
TC_1	Data Upload Dataset File Path	File uploaded	Successfully	Pass
TC _2	Data Cleaning	Raw Dataset	Cleaned Data	Pass
TC _3	Data Preparation for Training	Dataset and Split-Ratio	Train-set and Test-setcreated successfully	Pass
TC _4	Model Construction and Training	Training Algorithm And Train-set	Model trained successfully using train-set	Pass
TC _5	Model Validation	Trained Model And Test-set	Display model validation parameters with its values	Pass
TC _6	Display Result	Model Performance Statistics	Classification accuracy and error rate with plot	Pass

CHAPTER-7

RESULT AND ANALYSIS

A result is the final consequence of actions or events expressed qualitatively or quantitatively. Performance analysis is an operational analysis, is a set of basic quantitative relationship between the performance quantities.

7.1 coding

Import Libraries

```
In [1]: import warnings
        warnings.filterwarnings("ignore")
        import pandas as pd
        pd.set_option("display.max_columns",None)
        import numpy as np
        from matplotlib import pyplot as plt
        %matplotlib inline
        import seaborn as sns
        from sklearn.preprocessing import MinMaxScaler
        from sklearn.model_selection import train_test_split
        from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
        import pickle
        from imblearn.over_sampling import SMOTE
        import os
        for dir_name,_,filenames in os.walk("input"):
            for filename in filenames:
                print(os.path.join(dir_name, filename))

        input\diabetes.csv
```

```
In [2]: # Data Loading

In [3]: df = pd.read_csv("input/diabetes.csv")
```

Data Preprocessing

Preliminary Analysis

In [4]:
```
df.shape
```

Out[4]:
```
(768, 9)
```

In [5]:
```
df.head()
```

Out[5]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Analysing Nullvalues

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

Exploratory Data Analysis

In [7]:
```
df['Outcome'].value_counts()
```

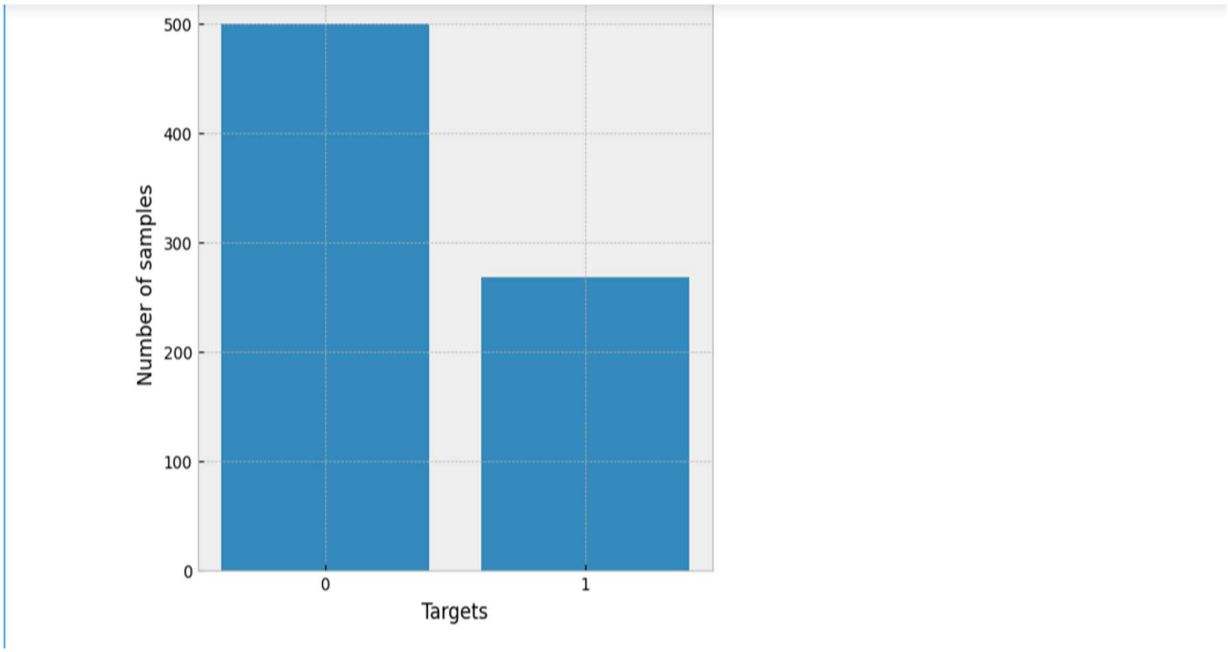
Out[7]:
```
0    500
1    268
Name: Outcome, dtype: int64
```

```
In [8]: chart_data = list(dict(df['Outcome'].value_counts()).values())
        chart_labels = [str(x) for x in list(dict(df['Outcome'].value_counts()).keys())]

        with plt.style.context(style="bmh"):
            plt.figure(figsize=(6,6))
            plt.bar(x=chart_labels, height=chart_data)
            plt.title(label="Feature used : Outcome\n Graph name : Bar-chart")
            plt.xlabel(xlabel="Targets")
            plt.ylabel(ylabel="Number of samples")
            plt.show()
```

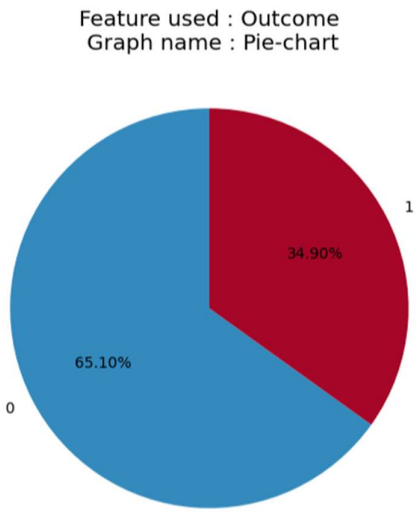
Feature Used: Outcome

Graph name: Bar-chart



In [9]:
```
chart_data = list(dict(df['Outcome'].value_counts()).values())
chart_labels = [str(x) for x in list(dict(df['Outcome'].value_counts()).keys())]

with plt.style.context(style="bmh"):
    plt.figure(figsize=(6,6))
    plt.pie(labels=chart_labels, x=chart_data, autopct='%.2f%%', startangle=90)
    plt.title(label="Feature used : Outcome\n Graph name : Pie-chart")
    plt.show()
```

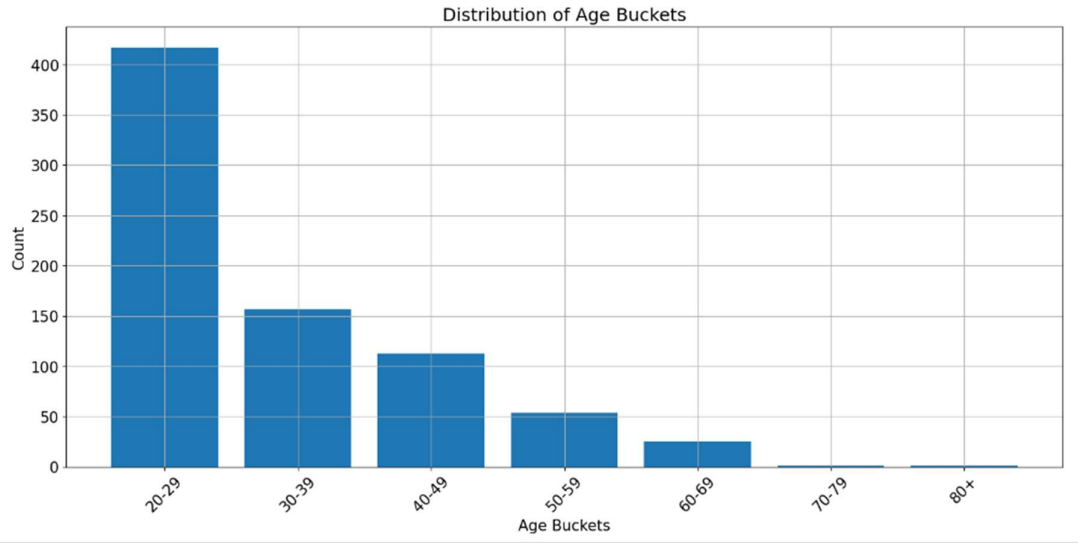


In [10]:
```
df.head()
```

Out[10]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

In [11]:
```
age_bins = [20, 30, 40, 50, 60, 70, 80, 90]
age_labels = ['20-29', '30-39', '40-49', '50-59', '60-69', '70-79', '80+']
df['age_bucket'] = pd.cut(df['Age'], bins=age_bins, labels=age_labels)
age_counts = df['age_bucket'].value_counts().sort_index()
plt.figure(figsize=(18,8))
plt.rcParams['font.size']=15
plt.bar(age_counts.index, age_counts.values)
plt.xlabel('Age Buckets')
plt.ylabel('Count')
plt.title('Distribution of Age Buckets')
plt.xticks(rotation=45)
plt.grid()
plt.show()
```



Normalization

In [12]:
```
scaler = MinMaxScaler()
scaler = scaler.fit(df.iloc[:,:-2])
scaled = scaler.transform(df.iloc[:,:-2])

data = pd.DataFrame(data=scaled, columns=df.iloc[:,:-2].columns)
data['Outcome'] = df.iloc[:,-2:-1]
data.head()
```

Out[12]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	0.352941	0.743719	0.590164	0.353535	0.000000	0.500745	0.234415	0.483333	1
1	0.058824	0.427136	0.540984	0.292929	0.000000	0.396423	0.116567	0.166667	0
2	0.470588	0.919598	0.524590	0.000000	0.000000	0.347243	0.253629	0.183333	1
3	0.058824	0.447236	0.540984	0.232323	0.111111	0.418778	0.038002	0.000000	0
4	0.000000	0.688442	0.327869	0.353535	0.198582	0.642325	0.943638	0.200000	1

In [13]:
```
with open(file="models/scaler.pkl", mode='wb') as file:
    pickle.dump(obj=scaler, file=file)
```

Oversampling

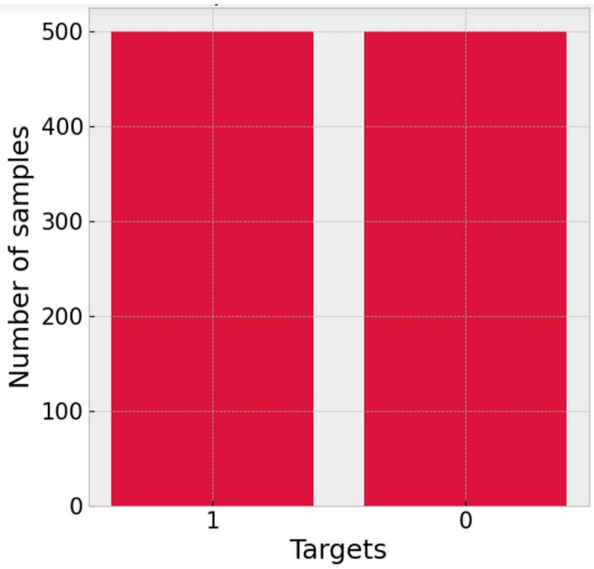
In [14]:
```
smote = SMOTE()
X_resampled, y_resampled = smote.fit_resample(data.iloc[:,:-1], data.iloc[:,-1:])
data = pd.concat(objs=[X_resampled, y_resampled], axis=1)
```

In [15]:
```
chart_data = list(dict(data['Outcome'].value_counts()).values())
chart_labels = [str(x) for x in list(dict(data['Outcome'].value_counts()).keys())]

with plt.style.context(style="bmh"):
    plt.figure(figsize=(6,6))
    plt.bar(x=chart_labels, height=chart_data, color='crimson')
    plt.title(label="Feature used : Outcome\n Graph name : Bar-chart")
    plt.xlabel(xlabel="Targets")
    plt.ylabel(ylabel="Number of samples")
    plt.show()
```

Feature Used: Outcome

Graph name: Bar-chart



In [16]:
```
data['Outcome'].value_counts()
```

Out[16]:
```
1    500
0    500
Name: Outcome, dtype: int64
```

Data Splitting

In [17]:
```
X = data.iloc[:, :-1]
X.head()
```

Out[17]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	0.352941	0.743719	0.590164	0.353535	0.000000	0.500745	0.234415	0.483333
1	0.058824	0.427136	0.540984	0.292929	0.000000	0.396423	0.116567	0.166667
2	0.470588	0.919598	0.524590	0.000000	0.000000	0.347243	0.253629	0.183333
3	0.058824	0.447236	0.540984	0.232323	0.111111	0.418778	0.038002	0.000000
4	0.000000	0.688442	0.327869	0.353535	0.198582	0.642325	0.943638	0.200000

In [18]:
```
y = data.iloc[:, -1:]
y.head()
```

Out[18]:

	Outcome
0	1
1	0
2	1
3	0
4	1

In [19]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=True, stratify=y)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(800, 8) (200, 8) (800, 1) (200, 1)
```

Algorithm:1 RandomForestClassifier

In [20]:
```
from sklearn.ensemble import RandomForestClassifier
RF_model = RandomForestClassifier()
RF_model = RF_model.fit(X_train.values, y_train.values.ravel())
```

In [21]:
```
RF_pred = RF_model.predict(X_test.values)
print(RF_pred.tolist())
```

```
[1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1,
0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0,
1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0,
0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0,
1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0]
```

Result Analysis

Accuracy Score

In [22]:
```
RF_accuracy = accuracy_score(y_true=y_test.values.ravel(), y_pred=RF_pred)
print(f"Validation accuracy of RandomForestClassifier model : {RF_accuracy*100.0:.2f}%")
```

```
Validation accuracy of RandomForestClassifier model : 80.50%
```

Classification Report

In [23]:
```
print(classification_report(y_true=y_test.values.ravel(), y_pred=RF_pred, target_names=['Positive', "Negative"]))
```

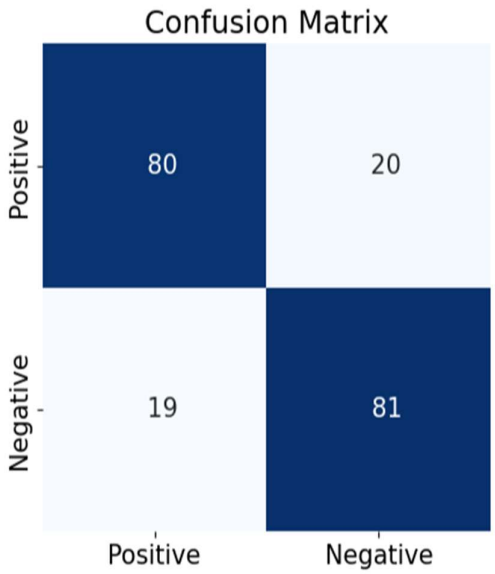
```
              precision    recall  f1-score   support

    Positive       0.81      0.80      0.80       100
    Negative       0.80      0.81      0.81       100

    accuracy                           0.81       200
   macro avg       0.81      0.81      0.80       200
weighted avg       0.81      0.81      0.80       200
```

Confusion Matrix

In [24]:
```
plt.figure(figsize=(5,5))
sns.heatmap(data=confusion_matrix(y_true=y_test.values.ravel(),
                                  y_pred=RF_pred),
                                  annot=True,
                                  cmap=plt.cm.Blues,
                                  fmt = "4d",
                                  xticklabels=['Positive', "Negative"],
                                  yticklabels=['Positive', "Negative"],
                                  cbar=False)
plt.title(label="Confusion Matrix")
plt.show()
```



In [25]:
```
with open(file="models/RandomForestClassifier_model.pkl", mode="wb") as file:
    pickle.dump(obj=RF_model, file=file)
```

Algorithm:2 SupportVectorClassifier

In [26]:
```
from sklearn.svm import SVC
SVC_model = SVC()
SVC_model = SVC_model.fit(X_train.values, y_train.values.ravel())
```

In [27]:
```
SVC_pred = SVC_model.predict(X_test.values)
print(SVC_pred.tolist())
```

```
[1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1,
0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0,
1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0,
0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0,
1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0]
```

Result Analysis

Accuracy Score

In [28]:
```
SVC_accuracy = accuracy_score(y_true=y_test.values.ravel(), y_pred=SVC_pred)
print(f"Validation accuracy of SupportVectorClassifier model : {SVC_accuracy*100.0:.2f}%")
```

```
Validation accuracy of SupportVectorClassifier model : 79.00%
```

Classification Report

In [29]:
```
print(classification_report(y_true=y_test.values.ravel(), y_pred=SVC_pred, target_names=['Positive', "Negative"]))
```

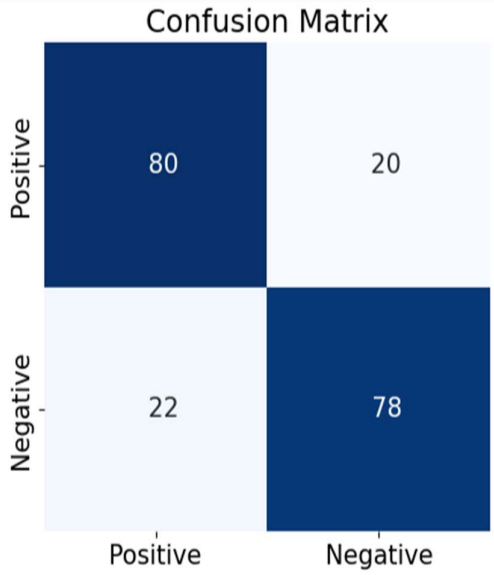
```
              precision    recall  f1-score   support

    Positive       0.78      0.80      0.79       100
    Negative       0.80      0.78      0.79       100

    accuracy                           0.79       200
   macro avg       0.79      0.79      0.79       200
weighted avg       0.79      0.79      0.79       200
```

Confusion Matrix

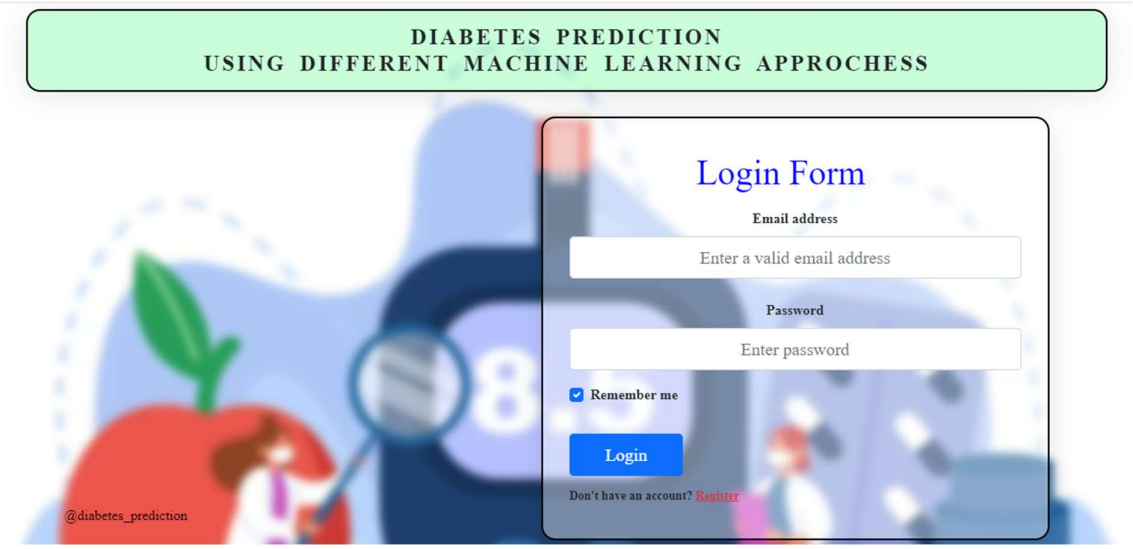
In [30]:
```
plt.figure(figsize=(5,5))
sns.heatmap(data=confusion_matrix(y_true=y_test.values.ravel(),
                                  y_pred=SVC_pred),
                                  annot=True,
                                  cmap=plt.cm.Blues,
                                  fmt = "4d",
                                  xticklabels=['Positive', "Negative"],
                                  yticklabels=['Positive', "Negative"],
                                  cbar=False)
plt.title(label="Confusion Matrix")
plt.show()
```



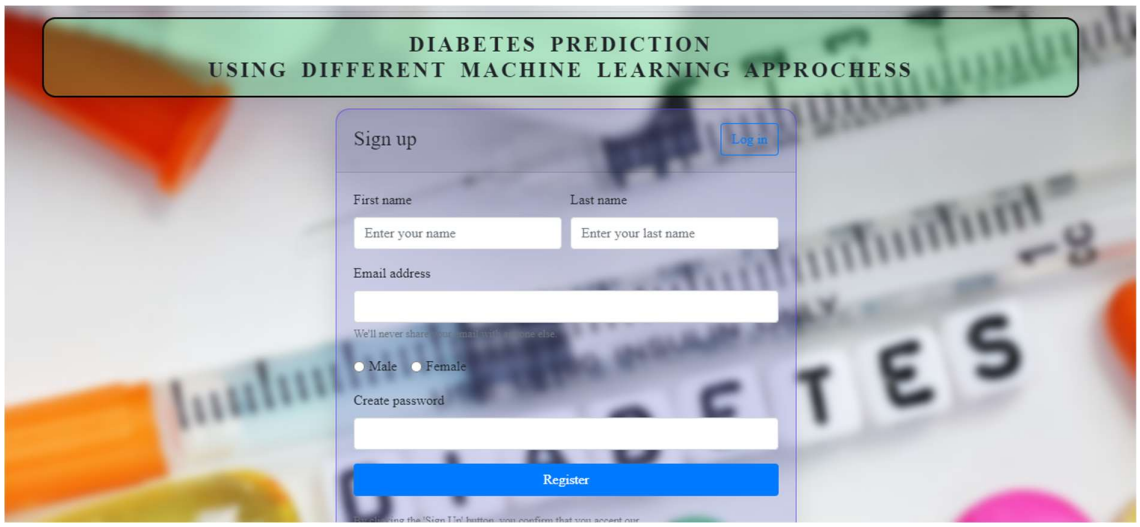
In [31]:
```
with open(file="models/SupportVectorClassifier_model.pkl", mode="wb") as file:
    pickle.dump(obj=SVC_model, file=file)
```

7.2 Screenshots

Login Form: A login form utilizes the credentials of a user, in order to authenticate their access. It generally consists of the typical username or email and password.

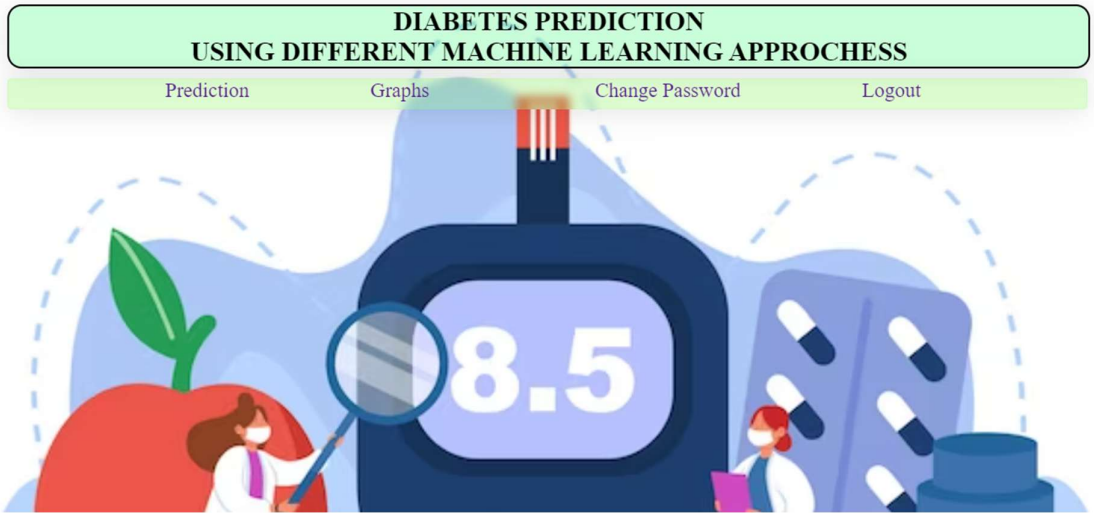


Register Form: A registration form is used to register or create a new account for a new user.

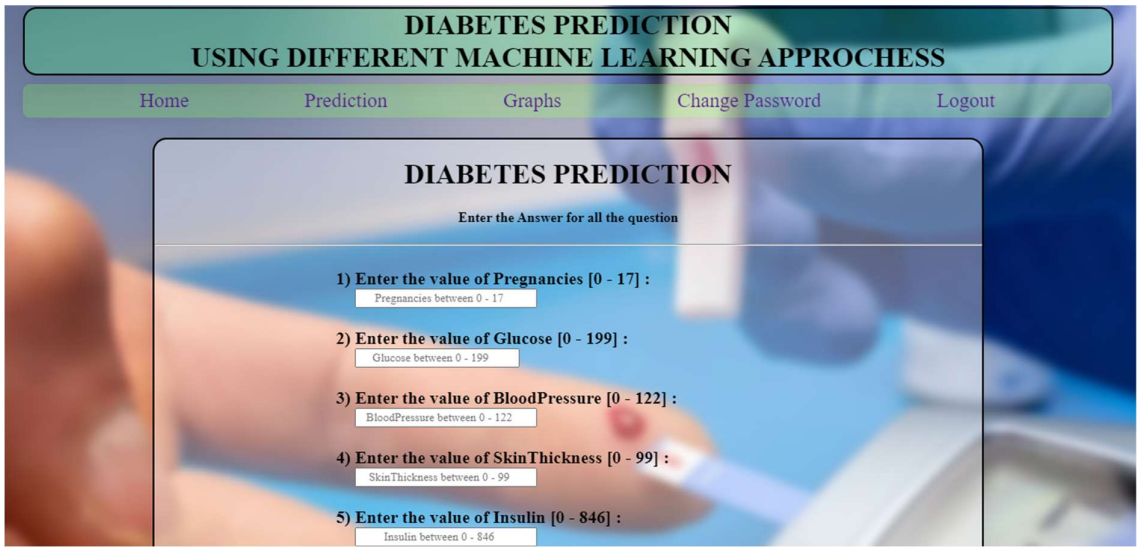


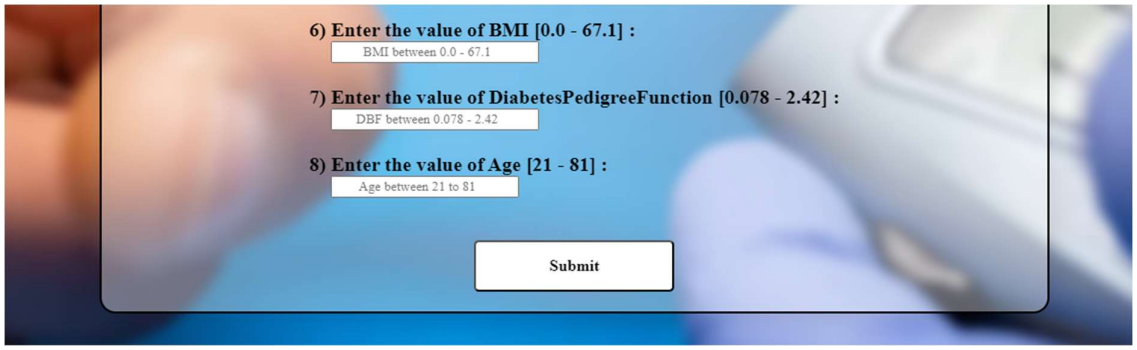
Home page: Here, user have four options to access. Prediction, Graphs, Change Password, Logout.

Logout: This form is used to exit a user account in a system.

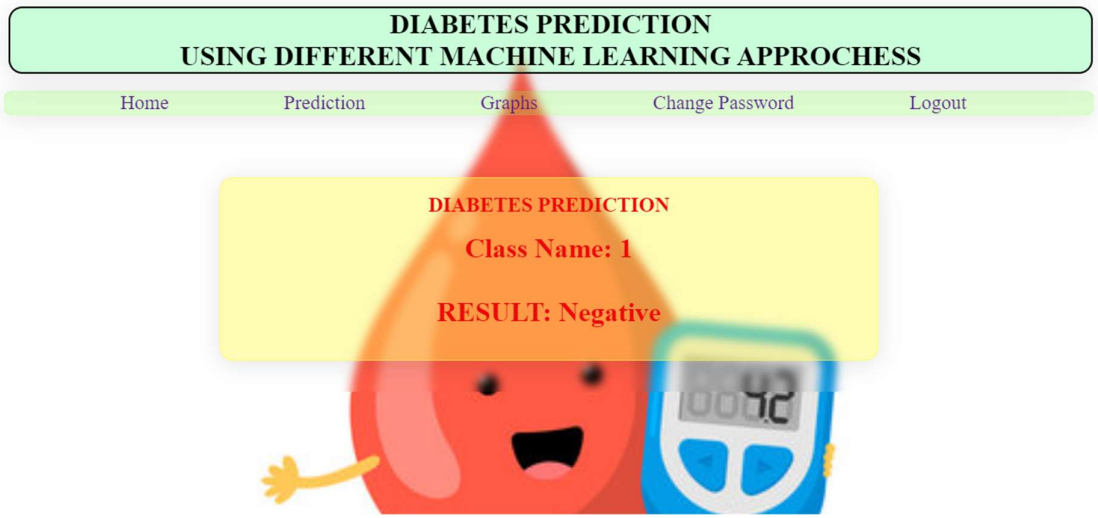


Prediction Form: This form predicts diabetes for that we have used 768 datasets with eight feature variables.

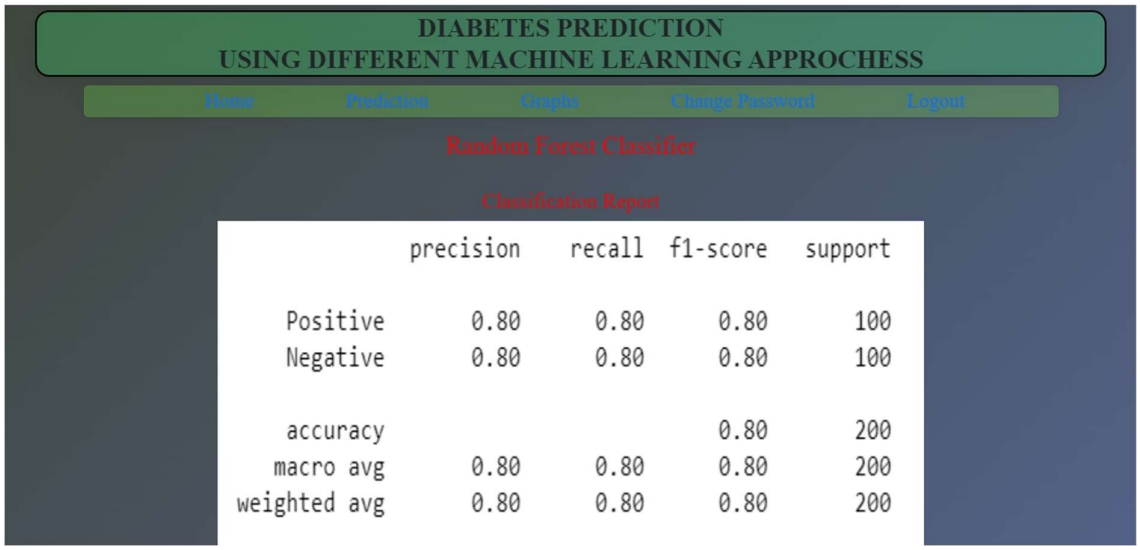




Result: In this form user can analyse the result of the patient individually whether the patient become diabetic or not (0 for positive and 1 for negative).

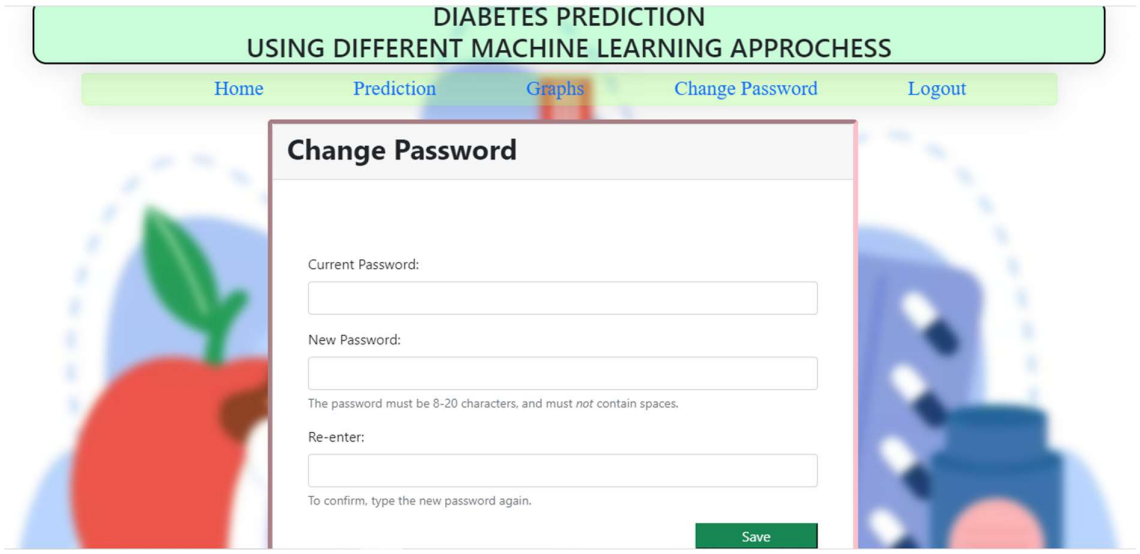


Methodology: Here, we used two machine learning algorithm Random Forest and SVM. Training set is used for building classification model and testing set is used for evaluating the performance of the model.



	precision	recall	f1-score	support
Positive	0.80	0.80	0.80	100
Negative	0.80	0.80	0.80	100
accuracy			0.80	200
macro avg	0.80	0.80	0.80	200
weighted avg	0.80	0.80	0.80	200

Change Password: In this form user can change their password, then it is impossible for someone else to gain access to the password.



CHAPTER-8

CONCLUSION

The machine learning methods can support the doctors to identify and prefer proper medication. We shall conclude that the improvement in classification accuracy helps to make the machine learning models get better results.

We have also seen that the accuracy of the existing system is less than 76.3% hence, we proposed to use a combination of classifiers known as Hybrid Approach. Hybrid approach takes advantage by aggregating the merits of two or more techniques. In future, if we get a large set of diabetic datasets, we can perform comparative analysis for analyzing the performance of each algorithm as well as the hybrid algorithms so that the best one can be applied for predictive analysis. A particular method to identify diabetes is not very sophisticated way for initial diabetes detection and it is not fully accurate for predicting diseases.

CHAPTER -9

REFERENCES

1)Veena Vijayan V. And Anjali C, Prediction and Diagnosis of Diabetes Mellitus, "A Machine Learning Approach" ,2015 IEEE Recent Advances in Intelligent Computational Systems (RAICS) | 10- 12 December 2015 | Trivandrum.

2) P. Suresh Kumar and V. Umatejaswi, "Diagnosing Diabetes using Data Mining Techniques", International Journal of Scientific and Research Publications, Volume 7, Issue 6, June 2017 705 ISSN 2250-3153.

3) Ridam Pal, Dr. Jayanta Poray, and Mainak Sen, "Application of Machine Learning Algorithms on Diabetic Retinopathy," 2017 2nd IEEE International Conference on Recent Trends in Electronics Information & Communication Technology, May19-20, 2017, India.

4) Berina Alic, Lejla Gurbeta and Almir Badnjevic, "Machine Learning Techniques for Classification of diabetes and cardiovascular diseases", 2017 6thMediterranean Conference on Embedded Computing (MECO), 11-15 JUNE 2017, BAR, MONTENEGRO.

5) Dr. M. Renuka Devi and J. Maria Shyla, "Analysis of Various Data Mining Techniques to Predict Diabetes Mellitus",International Journal of Applied Engineering Research ISSN 0973-4562 Volume 11, Number 1 (2016) pp 727-730 © Research India Publications. http://www.ripublication.com